

Bytehound

Serial Telemetry & Decoder Guide

Framed UART & Modbus RTU telemetry · live plots · TX commands · offline analysis

Table of Contents

1. Introduction & Getting Started	4
What is the Bytehound?	4
Application Data Flow & Architecture	4
Connecting Your Hardware	4
Auto-Reconnect	6
Active Polling (Auto-Fetch)	6
Unsolicited Data Detection	7
2. Core UI Panels & Features	8
The Main Data Table	8
Live Plot (Multi-Grid)	9
Grid Layout	9
Assigning Signals to Panels	10
Y-axis Scale Modes	10
View Modes	11
Controls	11
Trigger Configuration (Auto-Pause & Logging)	11

Bitfields Panel	12
Enums Panel	12
Raw Console	13
Activity Log.....	13
Fault Alarms.....	13
Status Bar (bottom-right counters)	14
Keyboard Shortcuts	15
The Menu Bar Reference	16
1. File Menu.....	16
2. Edit Menu	17
3. View Menu	17
4. Device Menu	18
5. Tools Menu	18
6. Help Menu.....	19
3. Controlling Your Hardware.....	20
TX Commands	20
Parameter Editor	21
How the value is encoded	21
Reading a TX packet (byte-by-byte)	22
Framed packet – variable length (no tx_pad_length)	22
Framed packet – fixed length (tx_pad_length set).....	22
Modbus RTU – single-register write (FC 06)	23
Quick reference – where each byte comes from.....	23
4. Logging and Data Analysis.....	24
Recording Logs.....	24
Decoded Log Cycle Buffer & Synchronization	25
Decoded Logger Unblocking for Mixed RX/TX Profiles.....	25
Transmitted (TX) Packet Decoding & Logging	25
Logging Settings	25
The Analysis Suite.....	26
Open and load logs.....	26

Layout and parameters.....	27
Plot behavior and view tools.....	27
Cursors and statistics.....	27
Export and sessions.....	28
Menu reference (quick guide).....	28
5. Customizing Your Data (The Excel Config).....	30
Config Validator CLI.....	30
Sheet 1 – Protocol.....	31
Fixed-length TX (tx_pad_length) – for buffer-full RX interrupts.....	36
Sheet 2 – Frames.....	37
Sheet 3 – Variables ★ (most important).....	39
Legacy – FrameConfig.....	42
Sheet 4 – Bitfields.....	44
Sheet 5 – Enums.....	45
Sheet 6 – CalcGroups.....	46
Sheet 7 – TxCommands.....	47
Sheet 8 – TxCommandFields.....	49
Sheet 9 – PollingSchedule.....	51
Sheet 10 – SerialDefaults.....	52
6. Appearance & Updates.....	54
Switching Light / Dark Theme.....	54
Check for Updates.....	54
Diagnostics & Troubleshooting.....	55
7. Application Limitations & Safety Caveats.....	55
Performance & Packet Processing Rates.....	55
Data Logging & Buffer Safety.....	55
Dock Window CPU Optimization (Data Gaps).....	56
Configuration Constraints.....	56

1. Introduction & Getting Started

What is the Bytehound?

Bytehound is a hardware-agnostic telemetry and control dashboard for serial devices. It reads framed UART/USB byte streams or Modbus RTU traffic, decodes the payload using a workbook or CSV configuration that you define, and displays the results in live tables, plots, logs, and command panels — all without writing Python code.

Application Data Flow & Architecture

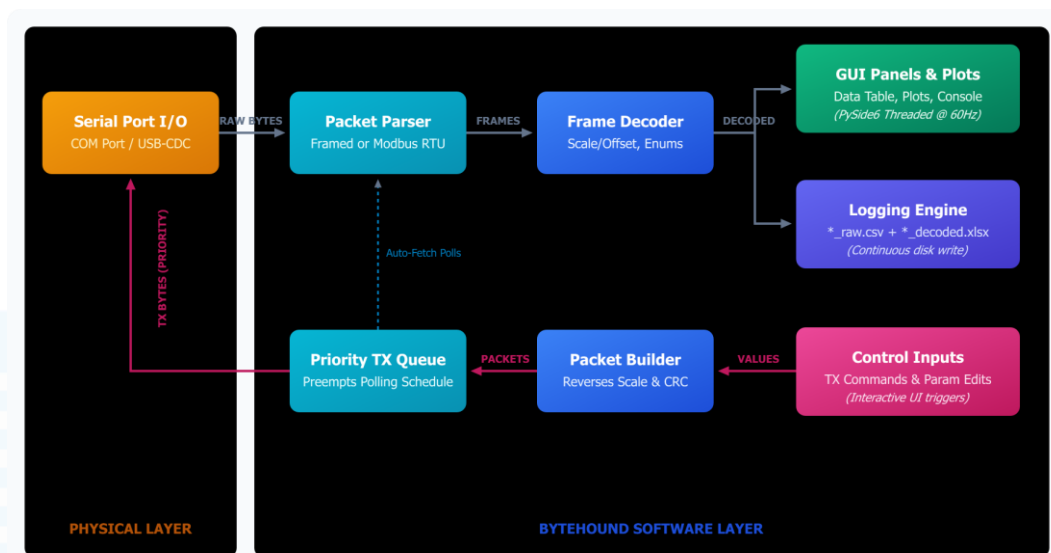


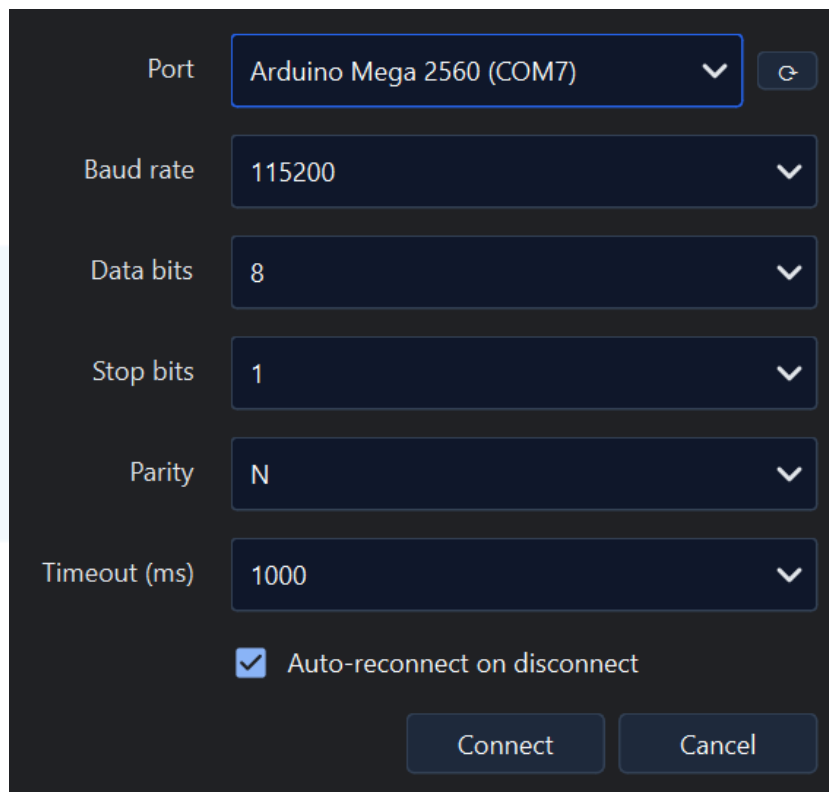
Figure: Application Data Flow & Architecture

Connecting Your Hardware

To get your hardware (NXP, STM32, ESP32, etc.) talking to the app:

1. Connect your hardware to your computer using a USB cable or Serial adapter.
2. Make sure your loaded config matches the bytes your device sends. For a framed protocol that usually means `[Header] + [ID] + [Length] + [Data] + [CRC]`; for Modbus RTU, set `parser_type = modbus_rtu`; for Waveshare USB-CAN-A adapters, set `parser_type = waveshare_can`. See [Section 5](#) for protocol details.
3. Launch **Bytehound**. A bundled example config loads automatically; use **Import Config** if you want to load your own Excel workbook or CSV config folder.
4. Click **Connect** in the toolbar. A **Connection Settings** dialog will appear:
 - Choose your **Connection Type** (**Serial**, **TCP Client**, or **UDP Client**) from the dropdown.
 - Configure the connection parameters depending on the selected type:

- **Serial:** Select the COM port (e.g. **COM7**). The ports dropdown list refreshes automatically when USB devices are plugged or unplugged. Set the Baud rate, Data bits, Stop bits, and Parity.
- **TCP Client:** Enter the target **Host / IP** and **Port** of the remote TCP server.
- **UDP Client:** Enter the target **Host / IP**, **Remote Port**, and an optional **Local Port** (binds locally, set to 0 for automatic port allocation).
- Set the common **Timeout (ms)** representing the read/response timing constraint.
- (Optional) Check **Auto-reconnect on disconnect** to have the app retry the connection automatically if the connection is dropped or a socket error occurs.
- Click **Connect** to establish the connection or socket wrapper.



The image shows a 'Serial Connection Settings Dialog Interface' with a dark background. It contains several settings: 'Port' is set to 'Arduino Mega 2560 (COM7)' with a refresh button; 'Baud rate' is '115200'; 'Data bits' is '8'; 'Stop bits' is '1'; 'Parity' is 'N'; and 'Timeout (ms)' is '1000'. There is a checkbox for 'Auto-reconnect on disconnect' which is checked. At the bottom are 'Connect' and 'Cancel' buttons.

Figure: Serial Connection Settings Dialog Interface

Once you click Connect:

- The connection status LED (the circular ● indicator on the left side of the bottom status bar) turns **green**.
- The toolbar Connect button changes to **Disconnect** with a **pink** background.

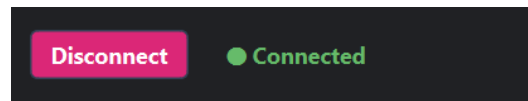


Figure: Toolbar Disconnect Button and Status Bar LED Indicators when Connected

Note If the device is unplugged while monitoring, Bytehound automatically stops logging, closes the worker, and returns the Connect button to its ready state. If the port stays open but no data arrives for several seconds, the LED turns amber and the status reads *Connected (No Data)*.

Auto-Reconnect

When **Auto-reconnect on disconnect** is checked in the Connection dialog, the app monitors the worker for unexpected disconnects (USB pull, device reset, TCP connection drop, socket read errors, or OS power management). On a drop it immediately schedules a reconnect attempt and uses **exponential back-off** so it doesn't hammer the serial/network stack:

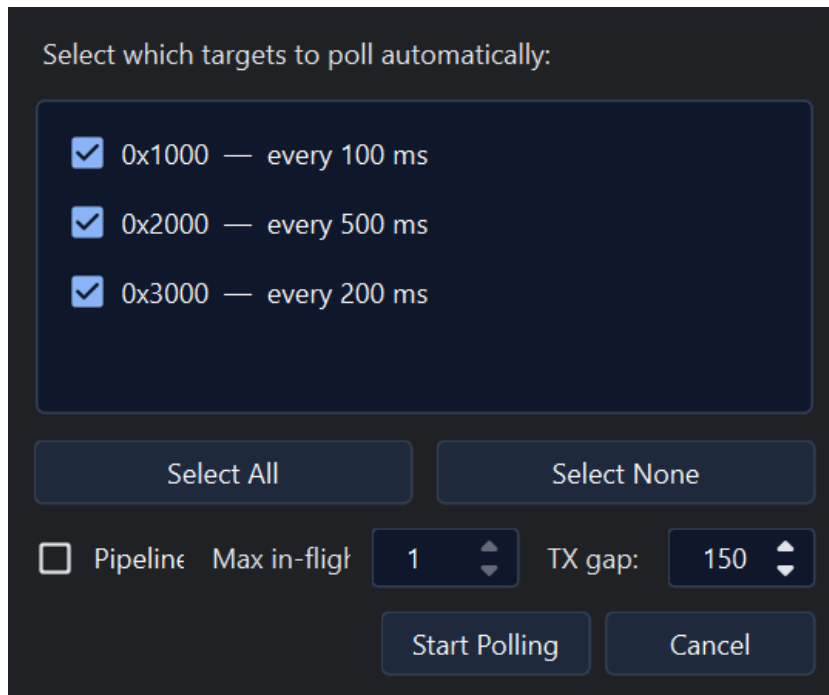
Attempt	Delay before retry
1	1 s
2	2 s
3	4 s
4	8 s
5+	16 s (capped)

Each attempt is logged to the **Activity Log** ([INFO] Auto-reconnect attempt N for COMx...). On success the app resumes normally, resets the counter, and logs [INFO] Auto-reconnect successful!. To cancel, click **Disconnect** or disable the checkbox before the next attempt.

Note Auto-reconnect does not re-start logging or Auto-Fetch automatically – it only re-opens the serial port. If you need logging to resume, start it manually after the reconnect notification appears in the Activity Log.

Active Polling (Auto-Fetch)

If your hardware only sends data when *asked* (request/response protocol), connect first, then use the **Start Auto-Fetch** toolbar button. A dialog lets you choose which configured targets are active. The background worker then sends requests based on your config's `polling_schedule` sheet.



Select which targets to poll automatically:

- ☒ 0x1000 — every 100 ms
- ☒ 0x2000 — every 500 ms
- ☒ 0x3000 — every 200 ms

Select All Select None

☐ Pipeline Max in-flight: 1 TX gap: 150

Start Polling Cancel

Figure: Start Auto-Fetch Polling Configuration Dialog

Pipelining and Timings: The Auto-Fetch dialog also allows you to configure **Pipeline poll requests**. By default, Bytehound sends one request and waits for its response (sequential). Pipelining allows multiple in-flight requests (controlled by *Max in-flight*), dramatically increasing throughput on hardware that supports it. You can also tune the **TX gap** (default 100 ms) which enforces a safe minimum spacing between consecutive outgoing polls.

When Auto-Fetch is active, the button text changes to **Stop Auto-Fetch** and turns **pink**. Click it again to stop. If your device streams data on its own, leave Auto-Fetch off.

Unsolicited Data Detection

In many telemetry setups, a connected hardware device streams data continuously and automatically (unsolicited) without needing to be polled. If you attempt to enable **Auto-Fetch** (polling) on such a device, it will cause transmission collisions, serial line corruption, and packets will be dropped.

To protect the communication link, Bytehound includes automatic unsolicited data detection. If the application receives valid telemetry packets while Auto-Fetch is disabled, it flags the link as containing unsolicited streaming data. If you then click **Start Auto-Fetch** (or press **F10**), Bytehound will show a **Potential Collision Warning** dialog:

⚠ Warning Potential Collision Warning: The connected device is already streaming data automatically (unsolicited). Enabling Auto-Fetch (polling) may cause transmission collisions and corrupt the data. Do you want to enable Auto-Fetch anyway?

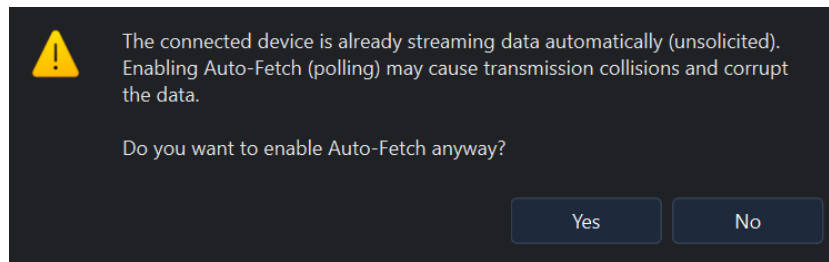


Figure: Potential Collision Warning Dialog

Clicking **No** aborts the Auto-Fetch request, ensuring the serial line is not saturated with conflicting commands. If your device supports both modes or has a specific hybrid query scheme, you can click **Yes** to override the warning and start polling anyway.

2. Core UI Panels & Features

The application is modular. The central Data Table stays in the middle, while dock panels such as Live Plot, Bitfields, Enums, Raw Console, and Activity Log can be dragged, stacked, resized, or hidden. Panel visibility is controlled under **View** → **Panels**, and **View** → **Reset Window Layout** restores the default arrangement.

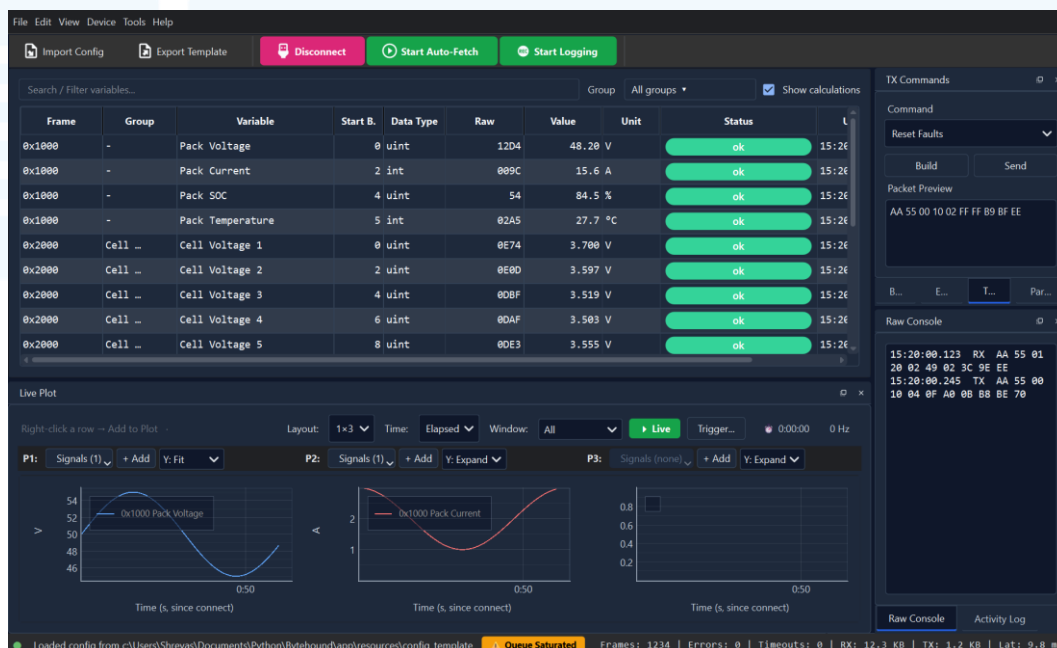


Figure: Bytehound Main Application Window Layout Overview

The Main Data Table

Located in the center of the screen, this read-only table displays all decoded variables in real time. The search box filters by text, the Group selector filters by configured signal group, and **Show calculations** controls whether calculated CalcGroups rows are visible. Columns include:

Column	Description
Frame	Frame ID in hex (e.g. 0x0010)
Group	Signal group from config, or the calculated group for CalcGroups rows
Variable	Signal name
Start B.	Start byte offset within the frame payload
Data Type	Decoded data type (e.g. uint16, float32)
Raw	Raw integer or float value before scaling
Value	Scaled engineering value, enum label, bitfield summary, or calculated value
Unit	Engineering unit (e.g. V, A, °C)
Status	Color-coded pill: OK, warnings, decode errors, enum labels, or active bitfield names
Updated	Timestamp of last received packet for this signal

Right-click any row to add that signal to the Live Plot or copy its value to clipboard. Use **Edit** → **Copy Table Snapshot** (Ctrl+Shift+T) to copy the full table as tab-separated text – paste directly into Excel or Sheets for a quick bench snapshot without setting up full logging.

Live Plot (Multi-Grid)

The Live Plot panel provides a fully flexible, multi-panel oscilloscope-style view of your telemetry data.

Grid Layout

Use the **Layout** dropdown in the plot toolbar to select how many subplots to display:

Layout	Panels	Best for
1×1	1	Focus on a single signal group
2×1	2 (stacked)	Compare two independent signal groups
1×2	2 (side by side)	Wide-screen dual view
1×3	3 (side by side)	Triple-wide comparisons
3×1	3 (stacked)	Vertical trend comparison
2×2	4	Full dashboard view
2×4	8	High-density telemetry overview
4×2	8	Tall, multi-lane overview

All panels share the same X-axis (time) – panning or zooming one panel moves all of them simultaneously.



Figure: Live Plot Panel in a Multi-Grid Layout

Assigning Signals to Panels

Each panel has its own **variable strip** (now laid out using grid alignment for consistent stretching across multi-grid layout changes) above it:

- Click **+ Add** next to a panel label to pick a signal from your loaded config.
- Each assigned signal appears as a coloured chip button (e.g. ● VoltageA ✕). Click the chip to remove it.
- Use the **Y-scale dropdown** on each panel to control how the Y-axis tracks data (see *Y-axis Scale Modes* below).
- Alternatively, **right-click** any row in the Data Table → *Add to Live Plot* → choose the panel number from the sub-menu.

Panel assignments and layout are automatically saved and restored between sessions.

Y-axis Scale Modes

Each panel strip has a small dropdown that picks how its Y-axis behaves on every redraw:

Mode	Behaviour	When to use
Y: Fit	Tight auto-fit with 5% padding. Axis rescales every 0.5 s to the visible data.	Default. Clean signals where you want the curve to fill the panel.

Y: Loose	Auto-fit with 25% headroom. Small noise stays inside the padding instead of triggering rescales.	Signals that wiggle a bit but you still want auto-tracking.
Y: Expand	Axis only grows, never shrinks. Once the bounds are large enough to contain the data, they stay put.	Noisy signals where Fit causes the axis to "breathe". Industry-standard noise killer (Vector CANalyzer's <i>Auto max</i> , LabVIEW's <i>Loose Fit</i>).
Y: Manual	Locked at a fixed range. Opens a dialog to type Y min / Y max; mouse pan/zoom still works and persists.	Signals with a known expected range (e.g. cell voltages 2.5 – 4.2 V).

You can also **right-click** a panel → *Set Y Range...* to type bounds directly (this also switches the panel to Manual mode).

View Modes

Mode	Indicator	Behaviour
 Live	Default	X-axis always shows $t=0 \rightarrow$ current time. The view expands automatically as data accumulates.
 Explore	After any pan/zoom	View is frozen. New data still plots but the window doesn't move. Hint shows:  <i>Live to resume</i> .

Controls

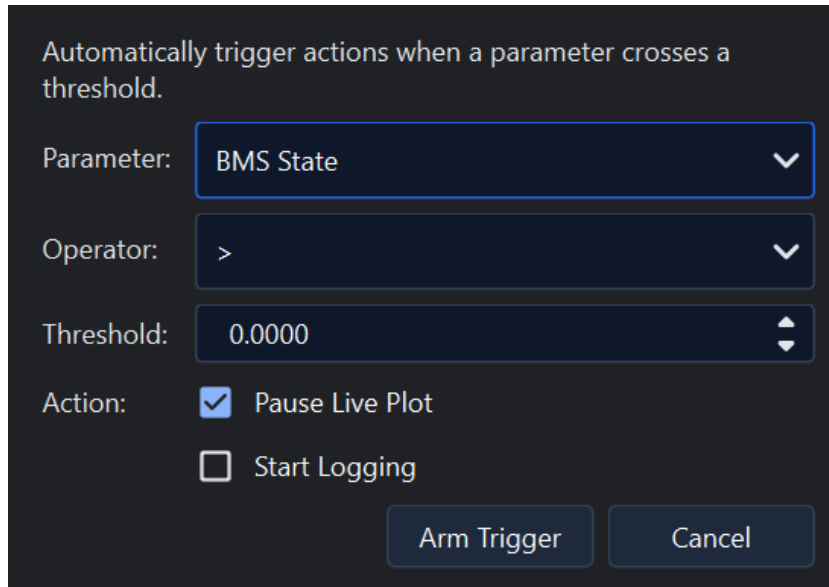
- **|| Pause / ▶ Live** button (or **Space**): Toggles freeze/scroll. Clicking **▶ Live** re-fits the Y axis and snaps X back to $0 \rightarrow$ now.
- **Time Mode dropdown**: Switch the X-axis labels between **Elapsed** (mm:ss since session start) and **Clock** (HH:MM:SS local wall-clock time). Your choice is saved between sessions.
- **Scroll wheel**: Zoom in/out on time axis → enters Explore mode.
- **Click + drag**: Pan the view → enters Explore mode.

Trigger Configuration (Auto-Pause & Logging)

Click the **Trigger...** button on the Live Plot toolbar to define an automatic action based on a signal's value crossing a threshold. When the configured condition (e.g., **Cell Voltage** > 4.2) is met in incoming telemetry:

- **Pause Plot**: Automatically freezes the plot (switches to Explore mode) so you don't lose the event off the left edge of the screen.
- **Start Logging**: Automatically begins recording a Raw + Decoded telemetry session, capturing the event as it happens without manual intervention.

The trigger disarms automatically after firing once. A **pink** active state on the Trigger button indicates it is armed.



Automatically trigger actions when a parameter crosses a threshold.

Parameter:

Operator:

Threshold:

Action: ☒ Pause Live Plot
☐ Start Logging

Figure: Live Plot Auto-Pause & Logging Trigger Settings Dialog

Note In Live mode the X-axis is anchored at **0 seconds** (session start) and expands to the current elapsed time. Plot samples are kept in a bounded in-memory history (default 6,000 points per signal), so very long sessions may show older time ranges with sparse or no plotted points even though the axis still starts at zero.

Bitfields Panel

If your hardware packs multiple Boolean flags into a single integer signal (like Fault states, Relay statuses, or Switch inputs), the **Bitfields** panel breaks them out into individual ON/OFF rows. The table columns are **Frame**, **Variable**, **Bit**, and **State**.

Enums Panel

Sometimes hardware sends state integers (e.g. 0 = OFF, 1 = PRECHARGE, 2 = RUN). The **Enums** panel shows the latest mapping for each configured enum signal with **Frame**, **Variable**, **Raw**, and **Label** columns.

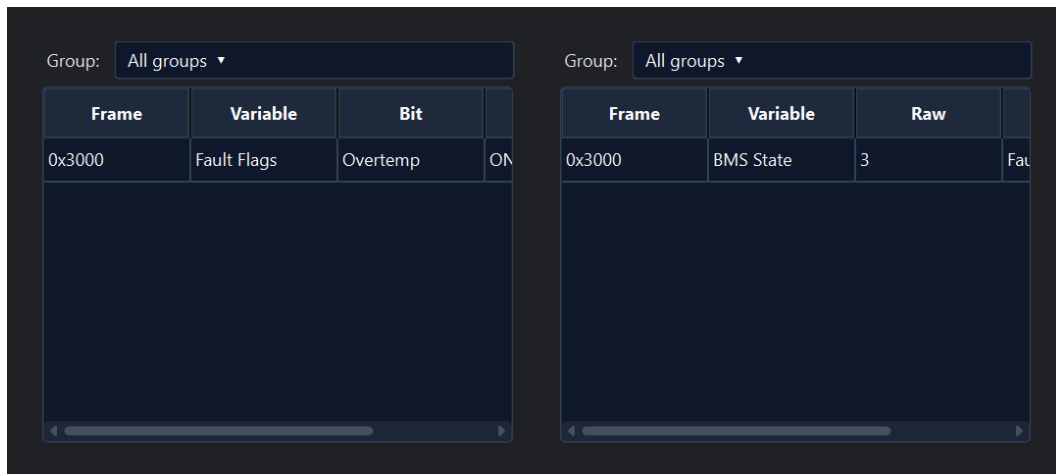


Figure: Bitfields and Enums Translation Dock Panels

Raw Console

The **Raw Console** shows timestamped hexadecimal bytes flowing into and out of the application in real time. Lines are tagged as RX, TX, or ERR, and decode problems appear as [decode] or [decode warning] messages. Use it to debug communication errors or verify packet structures.

Activity Log

The **Activity Log** tracks major application events: configurations loaded, connections established or lost, logging start/stop, and any error popups. Useful for post-mortem analysis of a test session.

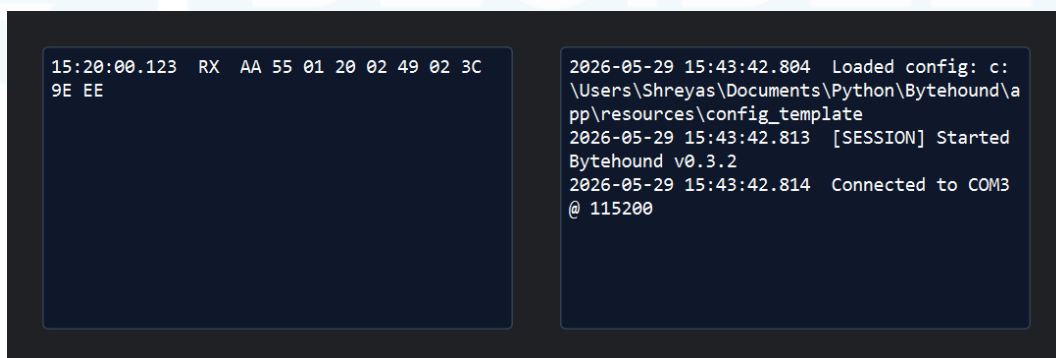


Figure: Raw Bytes Console and Activity Log Dock Panels

Frame Errors: If the application receives corrupted telemetry packets (e.g., CRC mismatch, invalid payload length, or bad footer), the exact reason will be displayed here as a [WARN] **Frame Error** message (e.g. [WARN] **Frame Error: CRC mismatch on frame 0x9000...**). To prevent the log from being flooded during a noisy connection, identical error messages are throttled to appear at most once per second.

Fault Alarms

Bytehound monitors all incoming decoded signals for the words **fault** or **error** (case-insensitive) in their Enum Labels or Bitfield states. If a fault is detected (e.g., a bit named "Overvoltage Fault" flips to ON),

Bytehound automatically logs an `[ALARM]` entry to the Activity Log and the Raw Console. This requires no setup—as long as your Variables configuration includes Enums or Bitfields with those keywords, the application will catch them.

Status Bar (bottom-right counters)

A single-line live readout sits in the bottom-right of the window with six fixed-width counters. They are updated continuously while connected and reset whenever you connect, load a new config, or click **Edit** → **Clear Console / Log**. The session clock (🕒) counts from the moment you connect – not from config load – so it always reflects actual device uptime. Reading the counters at a glance tells you whether the link is healthy and how the device is responding.

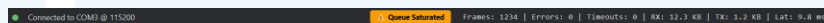
Frames: 1234 | Errors: 0 | Timeouts: 0 | RX: 12.3 KB | TX: 1.2 KB | Lat: 9.8 ms

Counter	What it means	What to watch for
Frames	Total RX frames received and parsed since the session started – both valid and CRC-failed frames are counted here.	Should climb steadily while the device is talking. A flat counter means the device has gone silent.
Errors	Wire-level parse failures such as CRC mismatch (including Modbus CRC) or footer mismatch. The frame had the right header/length framing, but the integrity check failed.	Should stay at 0 on a healthy link. A slowly climbing number suggests electrical noise, a bad cable, or a baud-rate mismatch. A fast climb almost always means the protocol settings (CRC type, byte order, header) in your config don't match the device.
Timeouts	Poll requests that did not get a response within the configured <code>timeout_ms</code> . Primarily relevant when Auto-Fetch is on; Modbus manual sends also wait for a response and can increment this counter.	0 on a request/response device that is keeping up. A few timeouts on first connect are normal (the device may still be booting). A continuously rising count means the device is missing requests – try a longer <code>timeout_ms</code> or a slower <code>interval_ms</code> .
RX	Total bytes received over the serial port since the session started, in B / KB / MB.	Roughly equal to <i>Frames</i> × <i>frame size</i> . A sudden plateau means the device stopped streaming.

TX	Total bytes transmitted by the app over the serial port – includes Auto-Fetch poll requests, TX-Command panel sends, and Parameter Editor writes.	0 if you are only listening. Climbs each time you send a command or while polling is active.
Lat	Time between the two most recently received frames, in milliseconds. Lower = faster device. This is wall-clock latency between consecutive RX frames, not request-response latency.	For a 10 ms streamer expect ~10 ms here. A spike to hundreds of ms means the device hiccuped; if it stays high the link is choked.

i Note Errors vs decode errors. The **Errors** counter only tracks wire-level CRC failures. Configuration-level problems – for example a frame ID with no matching **variables** row, a payload shorter than the configured signal layout, or an unsupported data type – are reported in the **Raw Console** as **[decode]** ... lines and are not counted here.

⚠ Warning ⚠ Queue Saturated badge. When the serial packet queue or the raw-log flush queue fills up (e.g. very high baud rate, slow disk, or window drag stalling the GUI), a yellow **⚠ Queue Saturated** badge appears to the left of the counters. Incoming packets or log entries are being dropped to protect memory. **Click the badge to dismiss it.** Investigate the cause – reduce baud rate, increase flush interval, or free up system resources – then clear the badge and monitor whether it reappears.



Connected to COM3 @ 115200 Queue Saturated Frames: 1234 | Errors: 0 | Timeouts: 0 | RX: 12.3 KB | TX: 1.2 KB | Lat: 9.0 ms

Figure: QStatusBar Indicators, Telemetry Counters, and Queue Saturated badge

Keyboard Shortcuts

All shortcuts are visible in the menu bar next to each action. Here is the complete reference:

Shortcut	Action	Notes
F9	Connect / Disconnect	Toggles the serial connection.
F10	Start / Stop Auto-Fetch	Opens the poll config dialog on first use.
Ctrl+L	Start / Stop Logging	Requires an active connection.
Ctrl+O	Import Config	Opens a file picker for .xlsx or CSV folder.
Ctrl+E	Export Template	Saves a blank config workbook.
Ctrl+K	Clear Console / Log	Resets all counters, clears decoded values and console.

Ctrl+Shift+C	Copy Value	Copies the selected table cell's value.
Ctrl+Shift+T	Copy Table Snapshot	Copies the full telemetry table as tab-separated text.
Ctrl+T	Analysis Suite	Opens the offline Analysis Suite window.
Ctrl+Q	Exit	Prompts if logging is active.
F1	View Documentation	Opens this manual in your browser.
Space	Pause / Resume Live Plot	Toggles between Live and Explore modes.

The Menu Bar Reference

The main window's menu bar provides direct access to all configuration, control, view, and utility functions of the application. Here is an in-depth breakdown of every menu, action, and setting to help you operate the application effectively, even if you are new to serial testing tools:

1. File Menu

The **File** menu is where you manage Bytehound's connection mappings (decoders) and exit the application safely.

- **Import Config (Ctrl+O)** *What it does:* Opens a file selection dialog to load a new protocol definition. You can select an Excel workbook (.xlsx or .xlsm) or a directory containing individual CSV sheets. *Why you need it:* Bytehound is generic; it doesn't know how your hardware packs its bytes until you tell it. Importing a configuration tells the app what each byte represents, what scales/offsets to apply, and how to format names. *Example:* You are testing a new Modbus-based temperature controller. You load `modbus_temp_sensor_map.xlsx`. Instantly, the main telemetry table updates to display rows for "Zone 1 Temp", "Heater Power %", and "Alarm Bitfield".
- **Export Template (Ctrl+E)** *What it does:* Generates a fresh, blank Excel workbook containing the template structure (10 required sheets with correct headers). *Why you need it:* If you are building a new sensor or controller and want to construct a telemetry profile, this gives you a valid starting point without needing to remember column names. *Example:* You want to monitor a custom DIY microcontroller project. You click Export Template, save it as `my_project_def.xlsx`, and open it in Excel to start typing in your signal names, offsets, and data types (e.g. `float32` at byte 4).
- **Exit (Ctrl+Q)** *What it does:* Closes the main application. *Safety Prompt:* If the application is currently recording telemetry logs, exiting will show a caution popup. This warns you that closing the app will immediately stop logging and finalize the logs. This prompt is critical because closing the app abruptly without stopping the log would result in losing all buffered decoded data.

2. Edit Menu

The **Edit** menu allows you to copy real-time data quickly or clear diagnostic clutter from the display.

- **Copy Value (Ctrl+Shift+C)** *What it does:* Copies the text contents of the currently selected cell in the center Data Table directly to the operating system clipboard. *Example:* You notice that the "Battery Voltage" signal wiggles to a peak value of 54.38 V and you want to document it. Highlight that cell in the table, press Ctrl+Shift+C, and paste the value into Slack, Notepad, or your bug tracking tool.
- **Copy Table Snapshot (Ctrl+Shift+T)** *What it does:* Captures the complete, real-time status of all decoded signals (columns and rows) in the main table and copies them as tab-separated values (TSV) to the clipboard. *Example:* You are running a bench test and want to quickly record the current status of all 50 cell voltages without starting a full session log. Press Ctrl+Shift+T, open Microsoft Excel or Google Sheets, click on cell A1, and press Paste. The entire table will populate instantly with aligned columns.
- **Clear Console / Log (Ctrl+K)** *What it does:* Resets all communication statistics (Frames, Errors, Timeouts, RX, TX, Latency) back to zero, clears all historical lines from the Raw Console and Activity Log text panes, and empties the main table's variables. *Why you need it:* If you have resolved a wiring issue that caused thousands of CRC errors, you want to clear the error count so you can monitor if any *new* errors occur. Pressing Ctrl+K gives you a clean slate.

3. View Menu

The **View** menu manages visual layout docking, themes, and configuration metadata.

- **Panels (Submenu)** *What it does:* Contains checkable options for each of the seven interface docks: *Live Plot*, *Bitfields*, *Enums*, *TX Commands*, *Parameter Editor*, *Raw Console*, and *Activity Log*. Unchecking a panel closes it; checking it opens it. *Why you need it:* Docks can be dragged, floating independently, or closed. If you accidentally close a panel (like the Live Plot), you can restore it here. *Example:* When doing read-only telemetry checks, you don't need to send commands. You uncheck *TX Commands* and *Parameter Editor* to hide them, freeing up screen space for the plot panel.
- **Reset Window Layout (Ctrl+Shift+R)** *What it does:* Restores all dock panels to their default positions and splits, and returns the window size to standard defaults. *Example:* You dragged the Live Plot to a second monitor and closed the console, and now your layout is disorganized. Pressing Ctrl+Shift+R snaps all panels back to their neat, default locations.
- **Config Info...** *What it does:* Displays a dialog summarizing the active configuration profile, showing the total number of parsed variables, valid frame IDs, CRC settings, and the serialdefaults configuration. Below the summary, the **Frame Format Diagram** renders a graphical, byte-aligned layout of the on-wire packet structure. *Tabs and Options:* Hover over any field block in either tab to see its name, byte offset, and size in bytes.
 - **RX Frames / Registers:** Shows the byte layout of incoming frames or Modbus read registers. Use the dropdown to inspect each frame's variables.

- **TX Commands:** Shows the byte layout of outgoing command packets, including static payload bytes, command fields, and any automatic packet padding. Use the dropdown to choose any configured command.
- **Theme (Submenu)** *What it does:* Switches the visual appearance of the application.
 - *Dark:* Best for low-light test benches or server rooms, reducing eye strain.
 - *Light:* Best for bright outdoor environments, test labs, or creating clean, high-contrast screenshots for technical documentation.
 - *System:* Automatically aligns the app with your Windows operating system light/dark theme settings.

4. Device Menu

The **Device** menu controls the physical hardware communication and data recording sessions.

- **Connect / Disconnect (F9)** *What it does:* Opens the Serial Settings dialog (allowing you to select COM Port, Baud rate, data/stop bits, parity, and timeout) and connects the serial interface. Toggles the connection on/off. *Example:* Plug in your USB-to-Serial adapter. Press **F9**, click Refresh, select **COM5**, set the baud rate to match your hardware (e.g. **115200**), and click Connect. The connection LED will turn green.
- **Start / Stop Auto-Fetch (F10)** *What it does:* Toggles active register polling on devices that follow a request/response (polled) protocol rather than streaming data continuously. *Example:* Your Modbus sensor requires a "Read Input Registers" command to be sent before it replies with the temperature. Enabling Auto-Fetch starts sending these commands automatically at intervals defined in the `polling_schedule` sheet.
- **Start / Stop Logging (Ctrl+L)** *What it does:* Opens the session recording dialog and begins logging data. *Modes:* **Note:** Logging works passively. You do **not** need to enable Auto-Fetch to log data if your device already streams it continuously. Enabling Auto-Fetch on a continuous streamer will cause collisions.
 - *Raw only:* Records raw incoming and outgoing hexadecimal bytes with timestamps directly to a `*_raw.csv` file. Safest for deep diagnostics.
 - *Decoded only:* Buffers decoded variables into memory and formats them as a wide Excel spreadsheet (`*_decoded.xlsx`) with metadata, written on Stop.
 - *Raw + Decoded:* Records both files simultaneously (highly recommended).

5. Tools Menu

The **Tools** menu hosts the post-test visual analysis suite and utility preferences.

- **Analysis Suite (Ctrl+T)** *What it does:* Opens a separate, non-modal plotting window for offline log analysis. *Why you need it:* It lets you load saved `*_decoded.xlsx` files from prior test runs, zoom in, place measurement cursors, apply smoothing filters, and overlay multiple runs to compare hardware behavior.

- **Logging Settings...** *What it does:* Opens the configuration dialog for internal rotating diagnostics and disk writing schedules. *Understanding the Settings:*
 - **Log level (DEBUG / INFO / WARNING / ERROR):** Controls how much diagnostic information the application writes to its internal `bytehound.log` file. If you set it to `INFO`, it only records major events (starts, connections). If you set it to `DEBUG`, it logs every transaction details, worker threads, and layout swaps—excellent for debugging connection drops or serial driver glitches.
 - **Flush interval (seconds):** Controls how frequently the Raw CSV logger writes buffered data from RAM to your disk. The default is `0.5 s`. Setting it to `0.0 s` forces an immediate disk write on every packet. This protects against data loss if the computer suddenly loses power, but it causes continuous disk activity which can degrade performance on older or slower drives.
- **Plot Settings...** *What it does:* Opens settings for the Live Plot panel. *Understanding the Settings:*
 - **Cap live plot memory history** – checkbox that enables a hard limit on how many samples the Live Plot stores per signal. When checked, set the **Memory cap limit** spinbox to the desired maximum (1 000 – 10 000 000 samples). When unchecked the history grows unbounded for the lifetime of the session; on long or high-rate sessions this can consume significant RAM.
 - **Default display window** – controls how wide the Live Plot X-axis is when first switched to Live mode. Options: *30 seconds, 1 minute, 2 minutes, 5 minutes (default), 10 minutes, 30 minutes, or All session*. Choosing *All session* always shows every sample from `t=0`; on long sessions this can make individual events hard to see – switch to a shorter window for real-time monitoring.

6. Help Menu

The **Help** menu provides manual documentation, software update tools, and debugging diagnostics.

- **View Documentation (F1)** *What it does:* Opens this manual (`index.html`) in your default web browser for quick offline reference.
- **Check for Updates** *What it does:* Safely connects to the online updates server to see if a new release is available. *Why it is safe:* It displays the version numbers and release notes first. If you confirm, it downloads the installer, computes its **SHA-256 cryptographic checksum**, and compares it with the server manifest. If the checksum matches, it runs the installer in silent mode to update your installation automatically. If it doesn't match, it blocks the install to protect you from corrupted downloads.
- **Copy Diagnostics** *What it does:* Automatically gathers details about your Windows version, Python version, PySide6/Qt versions, serial port configuration, data counters (frames, errors), and the last 200 lines of the application log. It copies this info to your clipboard. *Why you need it:* If you run into a bug, you can click this action and immediately paste the results into a bug report or support email, giving developers all the context they need to help you in one click.
- **Report Issue...** *What it does:* Opens a dialog to report software issues directly from within the application. *How it works:* Enter a title and description, and optionally include the tail of the application diagnostics log. When you submit, a background thread securely forwards your report along with complete runtime

context (OS, Python, Qt versions, active configuration path, connection status, packet counters) to our issue tracking server.

- **About Bytehound** *What it does:* Opens a window showing copyright notices, version info, credits, and links to the MIT License.

3. Controlling Your Hardware

The app isn't just for reading data – you can actively command your hardware over the same serial connection.

TX Commands

The **TX Commands** panel provides config-defined commands for sending predefined packets to your hardware. Choose a command from the selector, review the generated **Packet Preview**, then click **Send**. This is useful for triggering resets, clearing faults, or toggling modes.

Commands with input fields (defined in `tx_command_fields`) show an inline numeric form. The app validates ranges, applies scale/offset in reverse, appends the encoded field bytes after any static `payload_hex`, and builds the full packet automatically. Click **Build** to refresh the preview without transmitting.

Note **Send requires a connection.** Packet preview works from the loaded config even before a serial port is open, but **Send** will refuse to transmit until you connect.



The screenshot shows a dark-themed user interface for the TX Commands panel. At the top, there is a 'Command' label above a dropdown menu currently showing 'Set Voltage Limit'. Below the dropdown is a label 'Voltage Limit (V) (V)' next to a text input field containing the value '58.5'. Underneath the input field are two buttons: 'Build' and 'Send'. At the bottom of the panel, there is a 'Packet Preview' label above a text area displaying the hex string 'AA 55 00 20 02 DA 16 6D A1 EE'.

Figure: TX Commands Panel User Interface with Parameter Inputs and Hex Packet Preview

Parameter Editor

The **Parameter Editor** panel shows all signals in your config that are marked as writable (`read_write = RW` or `W`). For each signal you can:

- See the **Live Value** column update in real-time as packets arrive.
- Type a new value (in engineering units, e.g. `12.5` for 12.5 V) in the **Write** column's input field. Hover over the input to see the numeric range inferred from `min_value`, `max_value`, and the signal type.
- Press `Enter` or click the **Write** button. The app validates the range, reverses the scale/offset, encodes the raw bytes, appends the correct checksum, and transmits the packet immediately as a priority TX.



Figure: Parameter Editor Panel with Live Values and Editable Fields

How the value is encoded

The encoding formula is the exact inverse of how the signal is decoded:

```
raw = (your_value - offset) / scale
encoded_bytes = raw → data_type (uint8/16/32, int8/16/32, float32/64) → byte_order
```

Protocol	What gets sent
Framed	A complete framed packet is built for the signal's <code>frame_id</code> . The encoded bytes are placed at the exact <code>start_byte</code> offset (zero-padded payload). Header, length, CRC, and footer are all added automatically per the Protocol sheet.

Modbus RTU

An FC 06 (single-register) request is built when the encoded payload is 2 bytes; FC 16 (multi-register) is used for longer payloads. The raw value is encoded using the signal's scale, offset, and byte order, and CRC16/Modbus is appended automatically.

⚠ Warning Writes require an active serial connection. If no signals appear in the editor, all signals in your config are set to `read_write = R` (read-only). Update your Excel config to mark the appropriate signals as **RW** or **W**.

i Note Parameter Editor writes are recorded in the **Activity Log** with the signal name, engineering value, unit, and full raw hex. TX Command sends are also echoed in the Raw Console as **TX** lines.

Reading a TX packet (byte-by-byte)

Every TX line you see in the Raw Console – whether it came from a **TX Command**, the **Parameter Editor**, or an Auto-Fetch poll request – follows the same wire format defined by your Protocol sheet. Use this guide to decode any TX line.

Framed packet – variable length (no tx_pad_length)

Reference packet from the bundled template (Set_Voltage_Limit at 58.5 V on frame `0x0120`):

TX, AA 55 01 20 02 49 02 3C 9E EE

AA 55	← header (2 bytes, from Protocol.header)
01 20	← frame_id 0x0120 (big-endian, from frame_id_size + byte_order)
02	← length = 2 (payload bytes only, from length_meaning=payload_only)
49 02	← payload, little-endian uint16 = 0x0249 = 585 → 585 × 0.1 = 58.5 V
3C 9E	← CRC16 over header..payload (crc_type, crc_size, crc_byte_order)
EE	← footer (1 byte, from Protocol.footer)

Framed packet – fixed length (tx_pad_length set)

Same protocol, but with `tx_pad_length = 16` and a write of 123 to a uint16 signal on frame `0x0050`:

TX, AA 55 00 50 02 7B 00 00 00 00 00 00 00 00 00 00

AA 55	← header (2 bytes)
00 50	← frame_id 0x0050
02	← length = 2 (payload only – padding is not counted)
7B 00	← payload, little-endian uint16 = 0x007B = 123
00 00 00 00 00 00 00 00 00 00	← 9 bytes of zero-padding to reach tx_pad_length=16

⚠ Warning If your packet ends in zeros with **no CRC or footer visible** like the example above, that means your config has `crc_type = none` and an empty `footer`. If you expected a CRC and footer, open your Protocol sheet and check those two fields. With a CRC and footer configured, the last bytes of the frame would be the CRC + footer, and the zero-padding would sit *between* the payload and the CRC. `tx_pad_length` is the **total frame length on the wire**, including CRC and footer.

Modbus RTU – single-register write (FC 06)

With `parser_type = modbus_rtu` and `modbus_node_address = 1`, writing 123 to register address `0x0050`:

TX, 01 06 00 50 00 7B 99 8C

01	← node address (from <code>Protocol.modbus_node_address</code>)
06	← function code: 06 = Write Single Register
00 50	← target register address <code>0x0050</code> (big-endian, per Modbus spec)
00 7B	← value to write, big-endian <code>uint16 = 0x007B = 123</code>
99 8C	← CRC16/Modbus over preceding bytes, little-endian (per Modbus spec)

i Note Modbus byte order is fixed by the spec: addresses and values are big-endian, the CRC is little-endian. The Protocol sheet's `byte_order` fields are ignored for Modbus.

Quick reference – where each byte comes from

Byte(s)	Source	How to change it
Header	<code>Protocol.header</code>	Edit the Protocol sheet's header hex value.
Frame ID	Variable's <code>id_or_address</code> (or <code>TxCommands</code> row)	Set in Variables / <code>TxCommands</code> .
Frame ID size + byte order	<code>frame_id_size</code> , <code>frame_id_byte_order</code>	Protocol sheet.
Length	Computed from payload size	Controlled by <code>length_size</code> , <code>length_byte_order</code> , <code>length_meaning</code> .
Payload bytes	Encoded from your value via <code>raw = (value - offset) / scale</code>	Set <code>scale</code> , <code>offset</code> , <code>data_type</code> , <code>byte_order</code> on the variable.

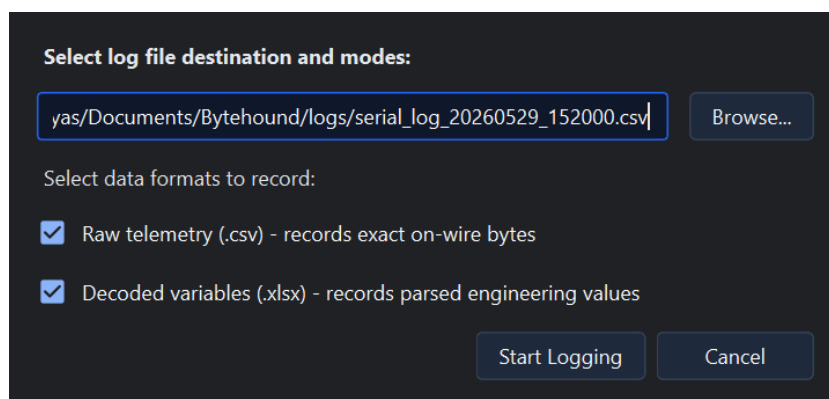
Zero-padding (if any)	Protocol.tx_pad_length	Set or clear tx_pad_length. Padding sits between payload and CRC.
CRC	Computed over crc_coverage region	crc_type, crc_size, crc_byte_order, crc_coverage.
Footer	Protocol.footer	Edit the Protocol sheet's footer hex value.

4. Logging and Data Analysis

Recording Logs

To save your telemetry session for later review:

1. Ensure you are connected and receiving data.
2. Click **Device** → **Start Logging** (or use the toolbar). Choose **Raw + Decoded**, **Raw only**, or **Decoded only**, then pick the base filename. The button turns **pink** while logging is active.
3. The default folder is `~/Documents/Bytehound/`. Depending on the selected mode, the app writes:
 - `*_raw.csv` – timestamped RX/TX hex frames with `timestamp`, `direction`, `hex`, and `delta_t_ms` columns. The byte-level capture of the session.
 - `*_decoded.xlsx` – Excel workbook with two sheets: **Metadata** (app, version, port, baud, config path, session start, logging mode, etc.) and **Data** (one wide row per complete poll cycle, with a per-frame block of `elapsed_ms` + `frame_id` + signals). Group calculations configured in the **CalcGroups** sheet are written without a frame ID prefix (e.g., `Cell Voltages min (V)` instead of `0x0100.Cells avg (V)`) so that they are clean and easy to identify.
4. When finished, click **Device** → **Stop Logging**. This finalises the decoded workbook.



Select log file destination and modes:

Select data formats to record:

☒ Raw telemetry (.csv) - records exact on-wire bytes

☒ Decoded variables (.xlsx) - records parsed engineering values

Figure: Start Logging Session Configuration Dialog

Note The raw CSV is flushed periodically (default 0.5 s, configurable under **Tools** → **Logging Settings**) and once more on close. The decoded `*_decoded.xlsx` is buffered in memory and saved on **Stop Logging** – if the app crashes before Stop, the decoded workbook is lost. A session config snapshot is saved beside the log so the same decoder can be re-applied later.

Decoded Log Cycle Buffer & Synchronization

The decoded Excel workbook (`*_decoded.xlsx`) stores telemetry in a wide format, where each row represents a complete cycle of all active frames. Individual frames arrive asynchronously and are stashed in an in-memory cycle buffer. When the **trigger frame** (the last RX-capable frame declared in the configuration) is received:

- Bytehound checks if all required frames for the cycle have been received.
- If the cycle is complete, a wide row is appended to the **Data** sheet containing timestamps and signal values for all frames.
- The cycle buffer is cleared upon trigger arrival to prevent carrying stale values forward to the next cycle.

Decoded Logger Unblocking for Mixed RX/TX Profiles

In telemetry configurations containing both incoming receive (RX) frames and outgoing transmit (TX) commands, requiring all frames to complete a cycle would cause logging to stall indefinitely (as commands are only sent on-demand). To prevent this, Bytehound implements **Decoded Logger Unblocking**:

- Transmit-only frames (those marked with `direction = tx` in the Frames sheet, or associated with TX commands) are dynamically excluded from the list of required cycle frames.
- The cycle is driven entirely by incoming receive-capable telemetry frames, ensuring that logging proceeds smoothly without stalling.

Transmitted (TX) Packet Decoding & Logging

To ensure that outbound controls and parameter modifications are fully captured, Bytehound hooks into raw wire transmissions to log outbound packets in the decoded stream on the fly:

- Whenever a command packet is sent (via the TX Commands panel, the Parameter Editor, or Auto-Fetch polling), Bytehound intercepts the raw bytes, parses them, and decodes the signals.
- These decoded TX values are stashed in the active cycle buffer.
- When the next trigger frame arrives, these stashed TX values are committed to the `*_decoded.xlsx` file along with the RX telemetry, creating a unified, chronological log of all input/output signals.

Logging Settings

Under **Tools** → **Logging Settings**, you can configure application-level diagnostic logging and data flushing:

- **Log level:** Sets the verbosity of Bytehound's internal rotating log file (`bytehound.log`). `INFO` is the default; use `DEBUG` if you are troubleshooting parse failures or protocol issues.
- **Flush interval:** Controls how frequently the raw `*_raw.csv` telemetry log writes to disk. The default is `0.5 s`. Set to `0.0` to flush on every write, which ensures no raw data is lost if the app crashes, though it may cause a slight performance penalty on slower disks. (Decoded `.xlsx` logs are always kept in memory and only written on Stop.)

The Analysis Suite

For deep-diving into saved test data and comparing multiple runs. The window is non-modal, so you can keep it open while the main app is running.

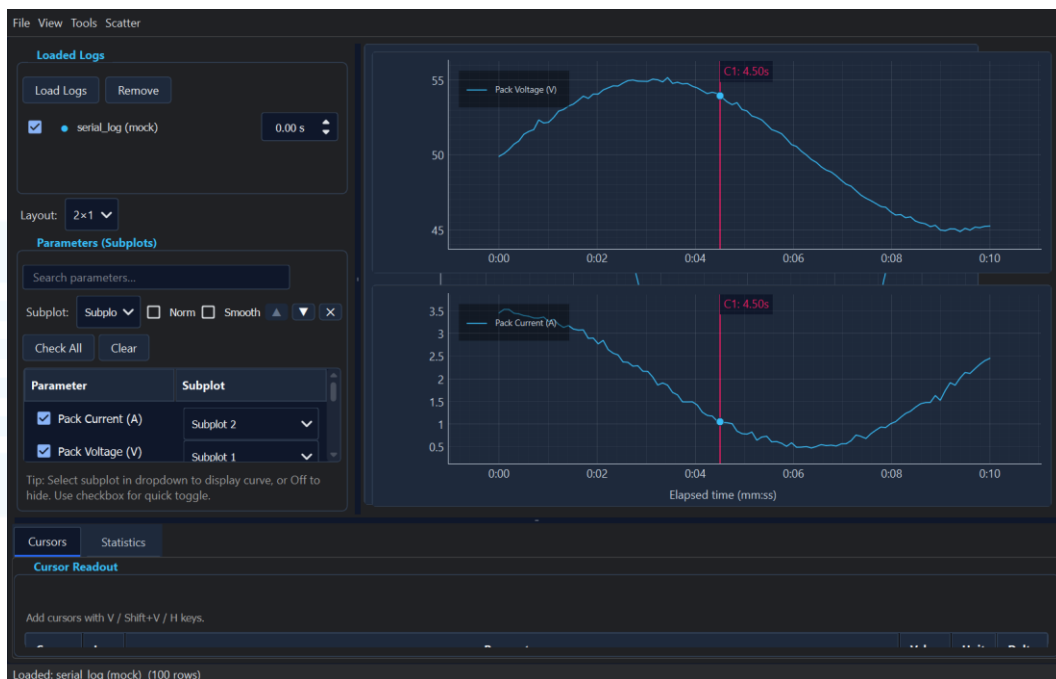


Figure: Offline Log Analysis Suite Interface

Open and load logs

1. Go to **Tools -> Analysis Suite**.
 2. Click **Load Logs** (or **File -> Load Logs...**) and choose one or more files.
 3. Supported formats: Bytehound decoded `.xlsx`, legacy decoded `.csv`, and external Dyno-style `.xlsx` exports.
- **Visibility:** use the checkbox beside each log to show or hide it.
 - **Color:** click the colored dot to change the log color.
 - **Offset:** use the time offset spin box (seconds) to align runs.

- **Remove:** the Remove button drops the most recently loaded log.

Layout and parameters

Use the **Layout** dropdown to choose the subplot grid. All subplots share a linked X-axis.

- **Check/clear:** the checkbox beside each parameter toggles its curve.
- **Search:** filter parameters with the search box; the hit counter shows matches.
- **Group/ungroup:** multi-select parameters, then use **Group** / **Ungroup** to control overlays.
- **Drag and drop:** drag parameters between subplots, or create a new empty subplot and drop into it.
- **Right-click:** move parameters to another subplot, normalize curves to 0-1, enable smoothing, or delete a subplot.

Plot behavior and view tools

- **Hover crosshair:** the status bar shows time and values under the cursor.
- **Multi-param encoding:** in multi-parameter subplots, color indicates parameter and line style indicates log; in single-parameter plots, color tracks the log.
- **Auto-fit Y: View -> Auto-Fit Y to Visible Range** re-scales Y after X zoom/pan (F).
- **Wall clock: View -> Wall Clock Time (X-Axis)** swaps elapsed time for HH:MM:SS when the log has a start timestamp.
- **Reset zoom: View -> Reset Zoom** or **Ctrl+0**.

Cursors and statistics

- **Vertical cursors:** V adds a cursor on the hovered subplot; **Shift+V** adds one across all subplots.
- **Horizontal cursor:** H adds a Y cursor at the current mouse position.
- **Edit:** drag cursors to measure deltas; **Delete** removes the selected cursor; **Tools -> Clear All Cursors** wipes them.
- **Cursors tab:** now shows a clean table with *Cursor*, *Log*, *Parameter*, *Value*, *Unit*, and *Delta*.
- **Vertical deltas:** when two vertical cursors are active, the Delta column shows C2 minus C1 for each curve.
- **Horizontal deltas:** when two horizontal cursors are on the same subplot, the Delta column shows H2 minus H1.
- **Cursor labels:** horizontal cursors are labeled in-plot as **H1: value**, **H2: value** and also appear as **H1 @ value** in the table.
- **Statistics tab:** shows min, P5, mean, median, P95, max, std, and n for the visible X-range of each curve.

i Note Statistics glossary: **P5 / P95:** 5th and 95th percentiles (values below/above which 5% of samples fall). **Std:** standard deviation, a measure of spread around the mean. **n:** number of samples in the current visible X-range.

i Note Example cursor table: Cursor Log Parameter Value Unit Delta C1 @ 1:14 serial_log Pack Voltage 53.81 V
C2 @ 2:16 serial_log Pack Voltage 48.73 V -5.08 H1 @ 48.7 serial_log Pack Voltage 48.70 V H2 @ 49.2 serial_log
Pack Voltage 49.20 V 0.50

Export and sessions

- **Sessions:** **File -> Save Session** stores layout, checked params, cursors, and view range; **Load Session...** restores them.
- **CSV export:** **File -> Export Visible Data as CSV...** writes a wide table over the current X-range. If the exported range is very dense (e.g. over 100,000 points), Bytehound will automatically prompt you to apply a decimation factor to keep the CSV file small and Excel-friendly.
- **Images/PDF:** **File -> Export Plots as Image...** or **Export Plots as PDF...**
- **Per-plot export:** right-click a subplot to export its visible range or copy the plot image.
- **Folders:** **File -> Open Data Logs Folder** and **Open Analysis Folder** jump to the saved output locations.

Menu reference (quick guide)

Short summary of where common actions live:

- **File:** load logs, manage sessions, export CSV/images/PDF, open output folders, close window.
- **View:** auto-fit Y, reset zoom, switch Cursors/Statistics tab, set smoothing window, toggle wall-clock time.
- **Layout:** group/ungroup parameters, add a new empty subplot.
- **Tools:** add or clear vertical/horizontal cursors.
- **Tools -> Import Schema Mapper...**: configure how the Analysis Suite reads external log files that don't follow Bytehound's default decoded `.xlsx` format. Three settings are available: Settings are persisted between sessions. Click **Save** to apply; changes take effect the next time you load a log file.
 - **Sheet Names** – comma-separated list of worksheet names to try when opening a workbook (e.g. `Data,Record,Sheet1`). The first matching sheet is used.
 - **Time Columns** – comma-separated list of column headers that represent elapsed time (e.g. `Elapsed(s),elapsed_ms`). The first matching column is used as the X-axis.
 - **Time Scale Mapping** – one `column_name: scale_factor` pair per line to convert the raw value to seconds (e.g. `elapsed_ms: 0.001` converts milliseconds to seconds). Leave blank for columns already in seconds.

Configure column/sheet headers for importing custom log files:

Sheet Names:

Time Columns:

Time Scale Mapping:

Figure: Import Schema Mapper Dialog

Additional Analysis tools include:

- **Scatter:** open the X-Y Plotter. The Plotter allows you to plot any two signals against each other (e.g., Voltage vs Current) across the visible time range, automatically applying 1D linear regression (best-fit line) and calculating the R-squared correlation coefficient. The X-Y Plotter window fully respects the active Light/Dark/System theme for consistent visual appearance.

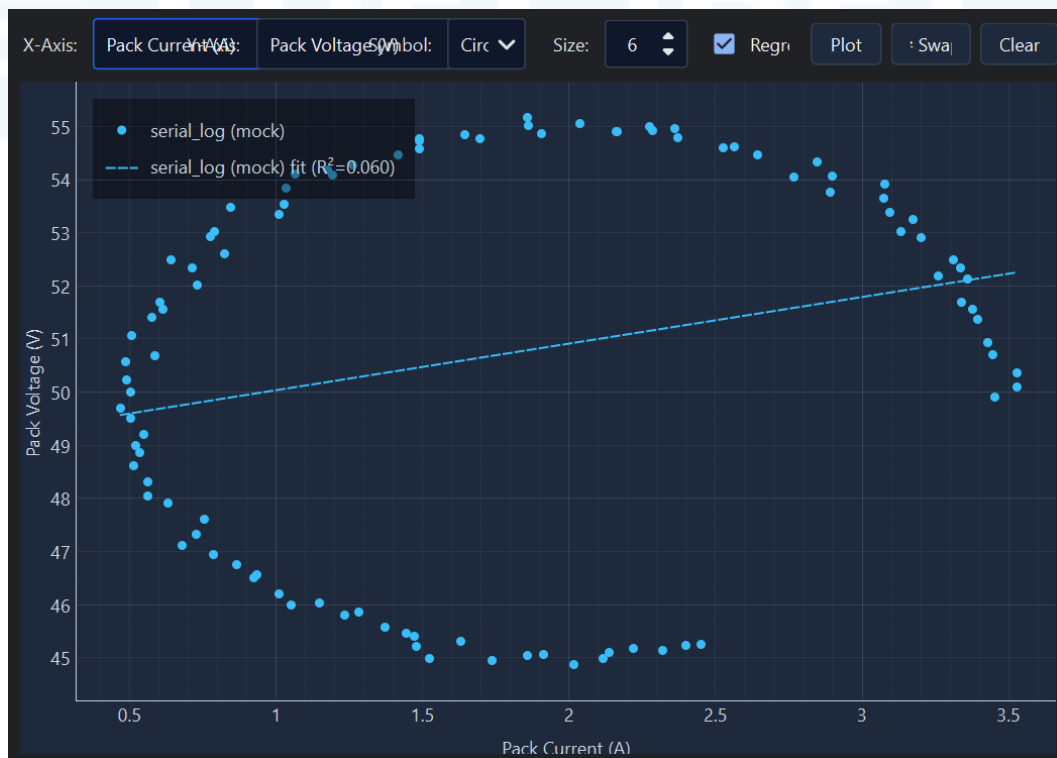


Figure: Scatter X-Y Plotter Window with Linear Regression

Note Quick tips: Hover any subplot to see live values in the status bar. Use **V** / **Shift+V** for vertical cursors and **H** for a horizontal cursor. Press **Delete** to remove the selected cursor. Right-click a subplot to export its visible range or copy an image. Right-click a subplot row in the parameter tree to normalize or smooth that subplot. Use the search box to filter parameters without losing your layout.

Tip Common workflows: **Align two runs:** load both logs, then adjust the time offset spin box on one log until key events line up. **Compare signal deltas:** add two vertical cursors and read the X and Y values in the Cursors tab. **Normalize mixed units:** group the parameters, then right-click the subplot header and enable Normalize (0-1). **Clean up noisy data:** enable smoothing on a subplot and set the window size under View -> Smoothing Window... **Export a report snapshot:** zoom to a range, then export CSV or PDF from the File menu.

5. Customizing Your Data (The Excel Config)

The Bytehound uses an Excel workbook to know *how to decode your hardware's raw bytes*, what to call each signal, how to scale it, and how to send commands back. You do not write any Python code – just fill in the sheets.

Note Quick start: Click **File** → **Export Template**, save the `.xlsx` file, open it in Excel, fill in the sheets described below, save, then click **File** → **Import Config** to load it. Changes take effect immediately.

Note Alternative CSV Folder Configuration: Instead of a single Excel `.xlsx` workbook, you can configure Bytehound using a directory containing individual CSV files. When importing, simply select **any CSV file inside the configuration folder**, and Bytehound will automatically read all related CSV files in that directory. The CSV filenames are case-insensitive and support normalization aliases: `protocol.csv` (maps to *Sheet 1 – Protocol*) `frames.csv` (maps to *Sheet 2 – Frames*) `variables.csv` or `frame_variables.csv` or `framevariables.csv` (maps to *Sheet 3 – Variables*) `bitfields.csv` (maps to *Sheet 4 – Bitfields*) `enums.csv` (maps to *Sheet 5 – Enums*) `calc_groups.csv` or `calcgroups.csv` (maps to *Sheet 6 – CalcGroups*) `tx_commands.csv` or `txcommands.csv` (maps to *Sheet 7 – TxCommands*) `tx_command_fields.csv` or `txcommandfields.csv` (maps to *Sheet 8 – TxCommandFields*) `polling_schedule.csv` or `pollingschedule.csv` (maps to *Sheet 9 – PollingSchedule*) `serial_defaults.csv` or `serialdefaults.csv` (maps to *Sheet 10 – SerialDefaults*) `frame_config.csv` or `frameconfig.csv` (maps to *Legacy – FrameConfig*) Each CSV file must contain a header row matching the exact column names documented for the corresponding sheet below.

Config Validator CLI

Bytehound can be run headlessly from the command line to validate configuration syntax without launching the GUI. Simply execute:

```
Bytehound.exe --validate config.xlsx
```

If the configuration schema is perfect (valid byte orders, correct types, no missing required fields), the CLI will print a success summary and exit with code `0`. If there are schema errors, it prints the errors and exits with code `1`, failing the build pipeline instantly.

The workbook contains the following sheets *in this order*. Each section below matches one sheet tab.

Sheet 1 – Protocol

Defines the wire-level packet structure. **Exactly one row must have `enabled = TRUE`**. Do not edit this sheet unless you designed the packet format.

Column	Type	Example	What it does
profile_name	text	BMS_v2	Free-text label for this protocol profile.
parser_type	enum	framed	framed = custom byte-framed protocol (default for most embedded MCUs). modbus_rtu = standard Modbus RTU – framing, FC codes, and CRC are handled automatically.
modbus_node_address	integer	1 (default)	Modbus slave address used as the first byte of every outgoing Modbus request. Only meaningful when <code>parser_type = modbus_rtu</code> ; ignored for framed. Defaults to 1 if blank or omitted.
header_hex	hex bytes	AA 55	Fixed start-of-frame marker the parser searches for. Space-separated hex bytes. <i>Ignored when <code>parser_type = modbus_rtu</code>.</i>
frame_id_size	integer	2	How many bytes carry the frame ID after the header. <i>Ignored for <code>modbus_rtu</code>.</i>

frame_id_byte_order	enum	little	big or little endian for the frame ID bytes.
length_size	integer	1	How many bytes carry the payload-length field. <i>Ignored for modbus_rtu (length is implicit in each FC).</i>
length_meaning	enum	payload_only (default)	What the length field counts on the wire. Four values supported: payload_only (just the payload bytes), frame_total (header + frame_id + length + payload + CRC + footer – the full frame size), header_to_crc (everything except the footer), payload_plus_crc (payload + CRC bytes). The parser and TX builder honour the chosen value end-to-end. <i>Ignored for modbus_rtu.</i>
crc_type	enum	crc16_modbus	Checksum algorithm: crc16_modbus, crc16_ccitt, crc32, or none. For modbus_rtu the CRC is always CRC16/Modbus regardless of this field.
crc_size	integer	2	How many bytes the CRC occupies at the end of the frame. <i>Ignored for modbus_rtu.</i>
crc_byte_order	enum	little	big or little byte order for the CRC field. <i>Ignored for modbus_rtu.</i>

footer_hex	hex bytes	<i>(leave blank)</i>	Optional end-of-frame marker. Leave blank if your protocol has no footer. <i>Ignored for modbus_rtu.</i>
raw_log_format	enum	hex <i>(default)</i>	Controls how the raw CSV logger formats the hex column. hex (default) writes space-separated uppercase bytes like AA 55 01 20. compact writes contiguous bytes like AA550120 – smaller files, faster to grep. Invalid values are rejected at load time.
inter_frame_delay_ms	integer	10	Minimum gap (ms) between consecutive TX requests. Default 10 ms.
tx_pad_length	integer	16 <i>(or blank)</i>	Forces every outgoing TX packet to be exactly this many bytes on the wire. Short commands are zero-padded between payload and CRC; CRC is computed <i>over</i> the padding so the receiver's check still passes. Leave blank to send variable-length packets. <i>Framed only – see Fixed-length TX below.</i>
length_byte_order	enum <i>(blank / big / little)</i>	<i>(leave blank)</i>	Byte order for the length field. Leave blank to inherit frame_id_byte_order – that covers almost every protocol. Only set this explicitly if your protocol uses multi-byte lengths in a different endianness from the frame ID.

crc_coverage	enum	header_to_payload <i>(default when blank)</i>	Which bytes the CRC is computed over. Four values supported: header_to_payload (header + frame_id + length + payload – the usual choice), frame_id_to_payload (skips the header), payload_only (just the payload), full_frame (everything except the CRC field itself, i.e. header + frame_id + length + payload + footer). The parser and TX builder use the same coverage so build/parse round-trips are byte-exact.
escape_mode	enum	none <i>(default when blank)</i>	Outer byte-stuffing / delimiter scheme applied <i>around</i> each Bytehound frame on the wire. Four values supported: none (current behaviour – frames are delimited by the header signature with no escaping), slip (RFC 1055: 0xC0 delimiter, 0xDB escape), hd1c (HDLC byte-stuffing: 0x7E flag, 0x7D escape with XOR 0x20), cobs (Consistent Overhead Byte Stuffing: 0x00 delimiter, code-prefixed chunks). The parser's outer un-framer runs first; the existing header / length / CRC / footer logic then processes the un-escaped inner frame.

enabled	TRUE/FALSE	TRUE	Only one row may be TRUE. The app uses that row and ignores all others.
---------	------------	------	---

i Note `length_meaning`, `crc_coverage`, and `escape_mode` all support multiple values now. Leave them blank to inherit the most common defaults (`payload_only`, `header_to_payload`, `none`) – these are what 99% of custom embedded protocols use.

⚠ Warning Interop caveat for the `length_meaning` and `crc_coverage` alternates. Bytehound's parser and TX builder agree end-to-end on all 16 combinations (32 build → parse round-trip tests cover them with and without a footer). What that proves is *internal* consistency: Bytehound can decode what Bytehound builds. It does **not** prove that Bytehound's interpretation of `frame_total` / `header_to_crc` / `payload_plus_crc` / `frame_id_to_payload` / `payload_only` / `full_frame` matches another vendor's protocol that happens to use the same name. Concretely: `frame_total` here counts *every* byte from the start of the header through the footer inclusive – header + `frame_id` + length + payload + CRC + footer. `header_to_crc` counts header + `frame_id` + length + payload + CRC (footer excluded). `payload_plus_crc` counts payload + CRC only. `full_frame` CRC covers header + `frame_id` + length + payload + footer (every byte except the CRC field itself). `frame_id_to_payload` CRC covers `frame_id` + length + payload (header excluded from CRC). `payload_only` (CRC) covers just the payload bytes. If you connect Bytehound to a real device that uses one of these names with even slightly different boundaries, the build/parse will silently mismatch the device – every TX frame will fail the device's CRC, every RX frame will fail Bytehound's CRC. **Test the first byte sequence on the wire** against your device's reference implementation before trusting these alternates in production.

⚠ Warning Same caveat for the `escape_mode` alternates. Bytehound implements SLIP per RFC 1055, HDLC byte-stuffing with the standard 0x7E flag / 0x7D escape / XOR 0x20 mapping, and COBS per Cheshire & Baker (1999). 134 round-trip tests cover every combination of `escape_mode` × `length_meaning` × `crc_coverage` × footer, plus targeted tests for the worst-case payloads (all-special bytes, frames chunked one byte at a time, three frames in one feed). Internal consistency is proven; **vendor interop is not**. Some HDLC implementations escape additional control bytes (0x11 / 0x13 XON/XOFF), some COBS variants insert a phantom zero, and at least one well-known "SLIP" device drops the leading 0xC0. Always verify the first wire bytes against your device's reference output before deploying.

✓ Tip Wire format (framed): [header] [frame_id] [length] [payload] [CRC] [footer?]
Wire format (modbus_rtu): [node_addr(1)] [FC(1)] [data bytes] [CRC16(2)] – FC03/FC04 read, FC06/FC16 write

i Note modbus_rtu is fully supported. Set `parser_type = modbus_rtu` and the parser and TX builder switch to standard Modbus RTU mode automatically. In this mode the `id_or_address` column in Variables and PollingSchedule is the Modbus register address (not a frame ID).

Note `waveshare_can` is fully supported. Set `parser_type = waveshare_can` to interface with Waveshare USB-CAN-A adapters over virtual serial ports. This parser automatically decodes both **Variable-Length Protocol** frames and **Fixed 20-Byte Protocol** frames (checksum validated). When active, the `id_or_address` column represents the CAN ID (e.g. `0x0123`). Columns like `header_hex`, `footer_hex`, and `length_size` can be omitted or left blank in your configuration, as they automatically fallback to Waveshare defaults. To build and send commands to a Waveshare adapter running in Fixed 20-Byte Protocol mode, set `waveshare_fixed_20_bytes` (or `waveshare_fixed`) to `True` in the Protocol configuration.

Fixed-length TX (`tx_pad_length`) – for buffer-full RX interrupts

Many microcontrollers receive serial data using a **buffer-full** or **DMA-complete** interrupt: the ISR fires once N bytes have arrived, and the firmware parses that fixed-size block. If your host sends commands of varying length, those firmwares either miss bytes, split commands across two interrupts, or sit waiting for bytes that never come. Setting `tx_pad_length` makes Bytehound transmit a constant-size frame for every command, regardless of payload.

Wire layout when padding is active:

```
[ header ][ frame_id ][ length ][ payload ][ 0x00 0x00 ... ][ CRC ][ footer? ]
      |----- padded region -----|
      |----- tx_pad_length total bytes -----|
```

- Padding bytes (`0x00`) are inserted *between* the payload and the CRC.
- The CRC is computed over header + frame_id + length + payload + padding, so the receiver's checksum still matches without any firmware change.
- The `length` field still reports the **real payload length** (not including padding). Firmware that uses buffer-full RX usually ignores this field anyway – it knows the frame size from the buffer size.
- Only TX is affected. Received frames are still parsed by header + length + CRC as normal.

How to pick the value: set `tx_pad_length` equal to your MCU's RX buffer size (or DMA transfer count). Example: an STM32 with `HAL_UART_Receive_IT(&huart1, buf, 16)` needs `tx_pad_length = 16`.

✓ **Tip Worked example.** Protocol with header = AA 55, frame_id_size = 2, length_size = 1, crc_size = 2, no footer, and `tx_pad_length = 16`: Command with 1-byte payload `0x42` for frame `0x0010` goes out as: AA 55 10 00 01 42 00 00 00 00 00 00 00 00 CRC_lo CRC_hi Total = 2 + 2 + 1 + 1 + 7 + 2 = **16 bytes**. The CRC covers the first 14 bytes, padding included.

⚠ **Warning** If a command is bigger than `tx_pad_length`, the app errors out instead of silently sending an oversized frame – you'll see TX frame (id=0x...) is N bytes but tx_pad_length is M. Either raise `tx_pad_length` or

shrink the command. Also, `tx_pad_length` must be at least `crc_size + len(footer)`, otherwise there's no room for the CRC and the app rejects the config.

Note **Not for Modbus RTU.** Modbus has its own fixed-per-FC framing – `tx_pad_length` is ignored when `parser_type = modbus_rtu`.

Bundled template content (the single enabled row that *File* → *Export Template* writes – blanks for `length_byte_order`, `crc_coverage`, `escape_mode`, and `tx_pad_length` mean "use the sensible default"):

```
profile_name           = Default
parser_type           = framed           ← `framed`, `modbus_rtu`, or `waveshare_can`
header_hex            = AA 55
frame_id_size         = 2
frame_id_byte_order   = little
length_size           = 1
length_meaning        = payload_only
length_byte_order     =                 ← inherits frame_id_byte_order
crc_type              = crc16_modbus
crc_size              = 2
crc_byte_order        = little
crc_coverage          =                 ← defaults to header_to_payload
footer_hex            = EE               ← matches the expected frame trailer byte
escape_mode           =                 ← defaults to none
raw_log_format        = hex              ← `hex` = spaced, `compact` = contiguous
inter_frame_delay_ms  = 10
tx_pad_length         =                 ← variable-length TX
modbus_node_address   = 1                ← Modbus slave address (ignored when parser_type =
framed)
enabled               = TRUE
```

Sheet 2 – Frames

Optional. Gives each frame ID a human-readable name and a **per-frame** expected payload length. If you omit this sheet, frame names are derived automatically from the Variables sheet.

Note **payload_length is per-frame, not global.** Each frame row declares its own expected payload size – one frame can carry 8 bytes while another carries 28 bytes. Leave `payload_length` blank for frames where you do not want length validation. Do **not** put a payload length in the Protocol sheet; that sheet only describes the wire framing (header, CRC, etc.).

Column	Type	Example	What it does
frame_id	hex <i>(required)</i>	0x0010	Frame identifier as sent by hardware. Can be 0x10, 10, or 16 (decimal).
frame_name	text <i>(required)</i>	BMS Status	Display name shown in the Data Table's Frame column.
payload_length	integer	8 <i>(blank = no validation)</i>	Expected payload size in bytes <i>for this frame only</i> . If the received payload differs, a decode warning appears in the Raw Console and Activity Log. Leave blank to skip validation for this frame.
direction	enum	rx tx <i>(default)</i>	Whether the frame can be received, transmitted, or both – controls UI gating . rx: telemetry-only – frame is parsed when received, but hidden from TX Commands and from the Parameter Editor (signals on rx-only frames cannot be written). tx: command-only – never expected on the RX wire. rx tx (default when blank): both directions. Case-insensitive. Invalid values are rejected at load time. Auto-created frames default to rx tx.
enabled	TRUE/FALSE	TRUE <i>(default)</i>	Set FALSE to completely ignore this frame – the row is skipped at load time and no signals from it are decoded.
description	text	Battery pack telemetry <i>(blank ok)</i>	Optional free-text note – not displayed in the app.

Bundled template content (what *File* → *Export Template* produces – same 3 frames the bundled Variables sheet references):

frame_id	frame_name	payload_length	direction	enabled	description
0x1000	BMS Status	8	rxtx	TRUE	Telemetry + Reset Faults TX target
0x2000	BMS Settings	8	rxtx	TRUE	Telemetry + Set Voltage Limit /
writable signals					
0x3000	Status Flags	8	rx	TRUE	Bitfield status register (telemetry only)

Sheet 3 – Variables ★ (most important)

Each row defines one numeric signal extracted from an incoming frame. The app reads bytes at the computed offset, applies scale and offset, and shows the result in the Data Table.

Column	Type	Example	What it does
id_or_address	hex (required)	0x0010	Frame ID (framed) or Modbus register address this signal lives in. If the ID is not declared in the Frames sheet, the loader auto-creates a frame named Node 0x<id> – convenient for Modbus configs that skip the Frames sheet entirely.
signal_name	text (required)	Pack Voltage	Unique name shown in the Data Table. Must be unique within each frame. For count > 1 the uniqueness check is applied <i>after</i> expansion, so Cell V with count = 12 becomes 12 distinct names.

data_type	enum (required)	uint16	Raw data type in the payload: uint8, int8, uint16, int16, uint32, int32, float32, float64 (with single and double supported as aliases for float32 and float64). Byte width is derived from the type and used to compute the next signal's start offset automatically.
count	integer	12 (default: 1)	Repeat count. count=12 on a signal named Cell V auto-creates Cell V 1 ... Cell V 12 at consecutive byte offsets – no copy-pasting 12 rows. Must be ≥ 1.
byte_order	enum	little (default)	little (LSB first, most embedded MCUs) or big (MSB first, network order). Validated at load – any other value is a hard error.
scale	number	0.001 (default: 1.0)	Multiply the raw integer by this. E.g. raw = 12500, scale = 0.001 → displayed as 12.5.
offset	number	-40 (default: 0.0)	Added after scaling. Formula: value = raw × scale + offset . E.g. temperature sensor with scale=0.1, offset=-40 converts 850 → 45 °C.
unit	text	V (blank ok)	Engineering unit shown in the Unit column (e.g. V, A, °C, %, rpm).
group	text	Cell Voltages (blank ok)	Group name used by CalcGroups for min/max/avg aggregation. Leave blank if not needed.

read_write	enum	R (default)	R = read-only. W = write-only. RW = readable and writable. Only W/RW signals appear in the Parameter Editor. Value is upper-cased on load.
min_value	number	0 (blank = no limit)	Minimum valid write value. The Parameter Editor rejects inputs below this. Leave blank for no limit.
max_value	number	5.0 (blank = no limit)	Maximum valid write value. Leave blank for no limit.
description	text	Total pack voltage (blank ok)	Optional note – not displayed in the app.
enabled	TRUE/FALSE	TRUE (default)	Set FALSE to hide this signal from all panels without deleting the row – the row is skipped entirely at load time.

i Note **Byte offsets are computed automatically.** Signals within a frame are packed in the order they appear in the sheet. You do not enter a start byte – the loader calculates it from each signal's `data_type` size.

i Note **Frame auto-creation.** If `id_or_address` refers to a frame ID that does not exist in the Frames sheet, the loader silently creates one named `Node 0x<id>`. This is the recommended workflow for Modbus configs – declare your registers in Variables only and omit the Frames sheet altogether.

Bundled template content (showing the key columns; `read_write`, `min_value`, `max_value`, `description`, `enabled` omitted for width):

id_or_address	signal_name	data_type	count	byte_order	scale	offset	unit	group
0x1000	Pack Voltage	uint16	1	little	0.01	0	V	
0x1000	Pack Current	int16	1	little	0.1	0	A	
0x1000	Pack SOC	uint8	1	little	1	0	%	
0x1000	Pack Temperature	int16	1	little	0.1	-40	°C	
0x2000	Cell Voltage	uint16	8	little	0.001	0	V	Cell
Voltages ← count=8 auto-expands to 8 signals								
0x3000	Fault Flags	uint8	1	little	1	0		
← see Bitfields sheet								
0x3000	BMS State	uint8	1	little	1	0		
← see Enums sheet								

0x2000 | Voltage Limit | uint16 | 1 | little | 0.01 | 0 | V |
← read_write=RW → Parameter Editor

Legacy – FrameConfig

An older schema with explicit `start_byte` and `byte_length`. Only used when the Variables sheet is absent. **Use Variables for all new configs.** Every column in this schema is required – there are no optional fields, no `count` auto-expansion, no `enabled` toggle, no per-frame `Frames` sheet (each row carries its own `frame_name`).

Column	Type	Example	What it does
<code>frame_id_hex</code>	hex <i>(required)</i>	0x10	Frame this signal belongs to. Accepts 0x10, 10, or 16 (decimal). Note: the column header is literally <code>frame_id_hex</code> , not <code>frame_id</code> – the loader will reject the sheet if this column is missing.
<code>frame_name</code>	text <i>(required)</i>	BMS Status	Display name for the frame. Repeated on every row of the same <code>frame_id_hex</code> ; the first occurrence is used to build the frame definition.
<code>signal_name</code>	text <i>(required)</i>	Voltage	Display name. Must be non-empty and unique within each frame – duplicates are a hard error at load time.
<code>start_byte</code>	integer <i>(required)</i>	0	Zero-based byte offset within the payload. Unlike the Variables sheet, offsets here are not auto-computed; you must specify every signal's exact position.

byte_length	integer (required)	2	Number of bytes to read. Must be 1..8 – anything outside this range is a hard error. For data_type = float, must be exactly 4 or 8.
endianness	enum (required)	little	little or big. Validated at load – any other value is a hard error.
data_type	enum (required)	uint	int, uint, or float. Coarser than the Variables sheet's uint16/int32/etc. – width comes from byte_length.
scale	number (required)	0.01	Multiplier applied to the raw value. Blank falls back to 1.0; non-numeric is a hard error.
offset	number (required)	0	Addend applied after scaling. Blank falls back to 0.0; non-numeric is a hard error. Formula: value = raw × scale + offset .
unit	text (required)	V	Display unit shown in the Unit column. May be an empty string but the column header itself must be present.

⚠ Warning Common pitfall: if your legacy CSV uses `frame_id` instead of `frame_id_hex` as the column header, the loader will reject the entire config with `missing required columns`. Rename the column to `frame_id_hex` or migrate to the modern Variables sheet.

Bundled template content (legacy schema for the same 0x1000 frame – every byte offset and length is explicit):

frame_id_hex	frame_name	signal_name	start_byte	byte_length	endianness	data_type	scale	offset	unit
0x1000	BMS Status	Pack Voltage	0	2	little	uint	0.01	0	V
0x1000	BMS Status	Pack Current	2	2	little	int	0.1	0	A
0x1000	BMS Status	Pack SOC	4	1	little	uint			

```
| 0 | %  
0x1000 | BMS Status | Pack Temperature | 5 | 2 | little | int |  
0.1 | -40 | °C
```

Sheet 4 – Bitfields

Breaks a single integer signal into named Boolean flags. Each bit appears as an ON/OFF indicator in the **Bitfields** panel.

Column	Type	Example	What it does
id_or_address	hex <i>(required)</i>	0x0020	Frame ID of the parent signal.
signal_name	text <i>(required)</i>	Fault Flags	Must exactly match a signal name in the Variables sheet. If the (id_or_address, signal_name) pair does not exist, the loader rejects the entire config at load time with unknown variable.
bit_index	integer <i>(required)</i>	0	Which bit to extract from the signal's raw integer value. Decoded as raw & (1 << bit_index), so bit 0 is the least-significant bit, bit 7 the MSB of a uint8, bit 15 the MSB of a uint16, etc. Valid range follows the parent signal's width (0..7 for uint8, 0..15 for uint16, 0..31 for uint32).
label	text <i>(required)</i>	Overvoltage	Name shown in the Bitfields panel for this bit.
active_text	text	FAULT <i>(default: ON)</i>	Text shown when the bit is 1. Blank cells also fall back to ON.

inactive_text	text	OK (default: OFF)	Text shown when the bit is 0. Blank cells also fall back to OFF.
---------------	------	-------------------	--

i Note Bit numbering recap. Bitfield rows operate on the *fully assembled raw integer* of the parent signal (after byte-order is applied), not on raw payload bytes. A `uint16` read as little-endian is reconstructed into a single 16-bit integer first; `bit_index = 15` then picks its most-significant bit regardless of which physical byte that bit came from.

Bundled template content (decodes the 8-bit `Fault Flags` register from frame `0x3000`):

id_or_address	signal_name	bit_index	label	active_text	inactive_text
0x3000	Fault Flags	0	Charging	YES	NO
0x3000	Fault Flags	1	Discharging	YES	NO
0x3000	Fault Flags	2	Balancing	YES	NO
0x3000	Fault Flags	3	Fault	FAULT	OK
0x3000	Fault Flags	4	Overvoltage	FAULT	OK
0x3000	Fault Flags	5	Undervoltage	FAULT	OK
0x3000	Fault Flags	6	Overtemp	FAULT	OK
0x3000	Fault Flags	7	Ready	YES	NO

Sheet 5 – Enums

Maps raw integer values of a signal to readable text labels. Results appear in the **Enums** panel and replace the raw number in the Data Table's Value column.

Column	Type	Example	What it does
id_or_address	hex (required)	0x0010	Frame ID of the parent signal.
signal_name	text (required)	State	Must exactly match a signal name in Variables. If the (id_or_address, signal_name) pair does not exist, the loader rejects the entire config at load time with unknown variable.
value	integer (required)	2	Raw integer value from the hardware. Decimal (2) or hex with prefix (0x02) – both work via <code>int(value, 0)</code> .

label	text <i>(required)</i>	RUN	Human-readable text shown when the signal equals this value. Empty labels are allowed but render as blank in the Enums panel.
-------	------------------------	-----	---

Bundled template content (decodes the `BMS State` signal from frame `0x3000` into one of five labels):

id_or_address	signal_name	value	label
0x3000	BMS State	0	Idle
0x3000	BMS State	1	Charging
0x3000	BMS State	2	Discharging
0x3000	BMS State	3	Fault
0x3000	BMS State	4	Service

i Note Duplicate values overwrite silently. Enum mappings are stored as `{value → label}` per signal, so if two rows share the same (`id_or_address`, `signal_name`, `value`) the second row's label wins. No warning is emitted – keep your `value` column unique within each signal.

Sheet 6 – CalcGroups

Computes aggregate statistics across decoded signals sharing the same `group` name. Results appear as extra rows at the bottom of the Data Table.

Column	Type	Example	What it does
group_name	text <i>(required)</i>	Cell Voltages	Must exactly match a non-empty group value in Variables. If the group does not exist, the loader rejects the entire config at load time with unknown group.
operations	pipe-list <i>(required)</i>	min max avg diff	Statistics to compute. Supported values are min, max, avg, sum, and diff (max – min). Separate with ; tokens are trimmed and lowercased. Any unsupported value rejects the config.

unit	text	V (blank ok)	Unit shown for the calculated rows. If left blank, the unit is automatically inherited from the signals in the group.
frame_id	integer / hex	0x0010 (blank = no frame filter)	Optional frame filter. Set this to run the calculation only for one frame; leave blank to run it for each decoded frame that contains signals in the group.
enabled	TRUE/FALSE	TRUE (default: TRUE)	Set FALSE to disable without deleting the row. Disabled rows are parsed, then skipped at load time.

⚠ Warning Frame scope pitfall. A blank `frame_id` does not combine values across multiple frames into one global statistic. It means "no frame filter": the calculation runs separately for each decoded frame that has signals in the matching group. Use a `0x` prefix for hex frame IDs here; bare values such as `10` are parsed as decimal.

Bundled template content (computes 4 stats across the 8 expanded `Cell Voltage 1..8` signals from frame `0x2000`):

group_name	operations	unit	frame_id	enabled
Cell Voltages	min max diff avg	V	0x2000	TRUE

How this links up:

1. In Variables, the `Cell Voltage` row at frame `0x2000` has `count = 8` and `group = Cell Voltages` — auto-expanded into `Cell Voltage 1 ... Cell Voltage 8`.
2. This CalcGroups row picks them up and adds four extra rows to the Data Table — `Cell Voltages min, max, avg, diff` — updated live with each `0x2000` frame.

Sheet 7 – TxCommands

Each enabled row creates one entry in the **TX Commands** panel. The app builds a packet from the static payload plus any matching TxCommandFields, then sends it to your hardware when you click **Send**.

Column	Type	Example	What it does
command_name	text <i>(required)</i>	Reset Faults	Command label shown in the TX Commands selector. Matching rows in TxCommandFields attach user-input fields to this command.
id_or_address	hex <i>(required)</i>	0x0090	Frame ID (or Modbus register address) of the outgoing packet. Must be non-empty; use a 0x prefix for clarity.
payload_hex	hex bytes	01 00 <i>(blank ok)</i>	Static payload bytes prepended before any TxCommandFields values. Leave blank if all payload bytes come from user-input fields. Invalid hex is reported when the command is previewed or sent.
description	text	Clears all active fault flags <i>(blank ok)</i>	Free-text note shown as the tooltip on the TX Commands selector – hover the currently selected command (or any entry in the open dropdown) to see this text. Keep it concise; one line is ideal.
enabled	TRUE/FALSE	TRUE <i>(default: TRUE)</i>	Set FALSE to hide the command without deleting the row. Disabled rows are skipped at load time.

⚠ Warning Duplicate names overwrite silently. Commands are stored by `command_name`, so if two enabled rows use the same name, the later row wins and no warning is emitted. Keep `command_name` unique.

i Note Reusing Addresses for Telemetry and Commands: Yes, you can (and often should) use the same `id_or_address` (frame ID or Modbus register address) in both the **Variables** sheet (Sheet 3) and the **TxCommands**

sheet (Sheet 7). In a custom **framed** protocol, telemetry variables you read from the device can share a frame ID with commands you write to the device. The device's firmware parses the frame ID to decide how to process it. In a **Modbus RTU** protocol, a register address (like `0x1000`) represents a specific physical memory location on the slave device. You read its value by defining it in the **Variables** sheet and setting a poll interval in the **PollingSchedule** sheet. You write a value to it by defining a command with the same address in the **TxCommands** sheet. Make sure that if you define a frame in the **Frames** sheet (Sheet 2) that is used for both RX and TX, its **direction** column is set to `rxtx` (which is the default). If it is set to `rx` (receive-only), the app will hide any TX commands using that frame ID to prevent accidental writes.

Bundled template content (one static-payload command and one parameterized command – the latter has matching rows in TxCommandFields):

command_name enabled	id_or_address	payload_hex	description
Reset Faults TRUE	0x1000	FF FF	Clears all active fault flags on the BMS
Set Voltage Limit input) TRUE	0x2000		Set the overvoltage threshold (requires field input)

Sheet 8 – TxCommandFields

Adds user-input numeric fields to a TX command. When a command has fields, the TX Commands panel shows an input form. The app encodes, scales, and appends those values after the command's static payload.

Column	Type	Example	What it does
command_name	text (required)	Set Current Limit	Links this field to a TxCommand. Must match an enabled TxCommands command_name exactly to appear in the form.
signal_name	text (required)	Current Limit	Label shown next to the input field. Keep this unique within a command so each input has its own value.

data_type	enum <i>(required)</i>	uint16	How the value is encoded in the outgoing packet: uint8, int8, uint16, int16, uint32, int32, float32, or float64 (with single and double supported as aliases for float32 and float64). Value is lowercased on load; unsupported types reject the config.
byte_order	enum	little <i>(default)</i>	little or big endian. Blank cells default to little; any other value rejects the config.
scale	number	0.1 <i>(default: 1.0)</i>	Encoding formula: raw = (user_value - offset) / scale . E.g. user enters 50.0 A with scale=0.1 → raw = 500. A zero scale loads successfully but the command cannot be built or sent.
offset	number	0 <i>(default: 0.0)</i>	Subtracted from the user value before dividing by scale.
unit	text	A <i>(blank ok)</i>	Shown next to the input field.
min_value	number	0 <i>(blank = no limit)</i>	App refuses to transmit if the entered value is below this.
max_value	number	200 <i>(blank = no limit)</i>	App refuses to transmit if the entered value is above this.

default	number	100 (blank ok)	Pre-filled value shown in the input box when the command form opens. Leave blank to require the user to enter a value before previewing or sending.
---------	--------	----------------	---

⚠ Warning Field matching is silent. Rows whose `command_name` does not match an enabled TxCommands row are parsed but never shown. If two fields on the same command reuse the same `signal_name`, the form labels collide and the later input value is reused for that name. Keep `command_name` valid and `signal_name` unique within each command.

Bundled template content (attaches one user-entered voltage field to the *Set Voltage Limit* command – encoded as `uint16 little` with scale 0.01):

command_name	signal_name	data_type	byte_order	scale	offset	unit	min_value	max_value	default
Set Voltage Limit	Voltage Limit (V)	uint16	little	0.01	0	V	0	60	55

A user typing 55.5 in the form sends $\text{raw} = (55.5 - 0) / 0.01 = 5550$ appended after the static payload (which is empty in this case).

Sheet 9 – PollingSchedule

Only needed if your hardware uses a **request / response** model – i.e. it only sends data when asked. Each row tells the app to periodically send a request packet and wait for a response.

Column	Type	Example	What it does
id_or_address	hex (required)	0x0010	Frame ID (or Modbus register address) to request. Must be non-empty; bare values such as 0010 are treated as hex, so use a 0x prefix for clarity.

interval_ms	integer (required)	100	How often to send the request, in milliseconds. Parsed as an integer and used to schedule the next poll after each request completes.
timeout_ms	integer (required)	50	How long to wait for the matching response before marking the request as timed out. Parsed as an integer; the current loader reads this column directly, so the header and value must be present.
enabled	TRUE/FALSE	TRUE (default: TRUE)	Set FALSE to pause polling for this frame without deleting the row. Disabled rows are skipped at load time.

Note Polling starts only when **Auto-Fetch** is enabled. Requests are sent round-robin across enabled targets; if a request cannot be built for the configured protocol, that target is disabled and reported once.

Warning If your hardware **streams data automatically** without being asked (e.g. sends packets every 50 ms on its own), leave this sheet **empty** and do **not** use Auto-Fetch. Polling an already-streaming device sends unnecessary requests and may corrupt the data stream.

Bundled template content (polls all three telemetry frames with per-frame intervals and timeouts):

id_or_address	interval_ms	timeout_ms	enabled
0x1000	100	50	true
0x2000	500	100	true
0x3000	200	50	true

Sheet 10 – SerialDefaults

Config-driven defaults for the Connection dialog. When the dialog opens, each field is filled from **QSettings (your last-used choice) if present, otherwise from this sheet**. That means a fresh install – or a freshly loaded config you've never connected with before – opens the Connect dialog already showing this config's expected baud rate, parity, etc.; subsequent runs keep whatever you explicitly chose. *Note: TCP and UDP connection*

defaults (Host, Ports, etc.) are managed and persisted directly via QSettings, as they represent host-level network configurations independent of the microcontroller's on-wire framing protocol defaults.

Column	Type	Example	What it does
baud_rate	integer (required)	115200	Baud-rate default for the Connection dialog. Used on first open; thereafter QSettings wins.
data_bits	integer (required)	8	Data-bits default for the Connection dialog. Combo offers 8 and 7.
stop_bits	number (required)	1 (blank: 1.0)	Stop-bits default for the Connection dialog. Combo offers 1, 1.5, 2. Blank cells fall back to 1.0.
parity	text (required)	N (blank: N)	Parity default for the Connection dialog. Combo offers N, E, O. Blank cells fall back to N; values are uppercased on load.
timeout_ms	integer (required)	100	Read-timeout default for the Connection dialog. Combo offers 20/50/100/250/500/1000 ms.

⚠ Warning Do not add a partially blank row. If this sheet is absent or empty, the loader uses internal defaults. If the sheet contains a row, all five column headers are required; `baud_rate`, `data_bits`, and `timeout_ms` must contain valid integers.

Bundled template content (standard 8-N-1 at 115 200 baud with a 100 ms read timeout – matches what the Connection dialog uses by default):

```
baud_rate | data_bits | stop_bits | parity | timeout_ms
115200    | 8          | 1         | N      | 100
```

6. Appearance & Updates

Switching Light / Dark Theme

Open **View** → **Theme** and pick one of:

- **Dark** – slate background with light text. Best for long monitoring sessions and dim labs.
- **Light** – white background with dark text. Best for outdoor / brightly lit environments and screenshots in reports.
- **System** – asks Qt to follow the operating system's light/dark preference.

The theme switch is instant. The app reapplies its window styling, rebuilds toolbar/menu icons, retints the live plot canvas, updates the Analysis Suite and the X-Y Plotter windows if open, and remembers your choice between launches.

Note Toolbar and menu icons are theme-aware. In light theme they are drawn with a dark glyph so they remain readable on the white menu background; in dark theme they switch to a light glyph automatically. No restart needed.

Warning System theme caveat. The base Qt palette follows the OS setting, but Bytehound's custom plot, icon, and Windows title-bar tinting are tuned for the explicit **Dark** and **Light** choices. If anything looks inconsistent in **System**, choose Dark or Light directly.

Check for Updates

Open **Help** → **Check for Updates**. The app will:

1. Read the local `version.json`, contact its configured update manifest over HTTPS, and compare the published version against your installed version.
2. If you are already on the latest build, show a brief *You are on the latest version* confirmation.
3. If a newer version is available, show the version number and release notes and ask whether you want to install it.
4. On confirmation, download the installer to your system temp folder with a progress dialog. You can cancel while the download is running – the partial file is automatically cleaned up.
5. Verify the installer **SHA-256 checksum** against the manifest before launching. If the digest does not match, or if the manifest does not provide a checksum, the install is refused; mismatched or partial downloads are deleted.
6. After the download is verified, ask one more time before launching the installer in silent mode. The app then exits so the installer can replace the running executable. When the installer finishes, relaunch Bytehound manually.

Note Update checks are entirely on-demand. The app never contacts the network on its own – nothing is downloaded or installed without your explicit click.

Warning If you see *"Failed to check for updates: connection timed out"*, the manifest server is unreachable from this machine. Check your firewall / proxy settings, or simply skip the update – the app continues to work normally.

Diagnostics & Troubleshooting

If you encounter issues, use **Help** → **Copy Diagnostics**. This generates a complete snapshot of your session state—including OS details, Bytehound version, Qt/Python versions, active config path, connection status, packet counters, and the tail of the internal application log. You can paste this directly into a bug report to provide all necessary context in one step without needing to manually hunt down the log file.

7. Application Limitations & Safety Caveats

Bytehound is designed to be highly flexible and performant. However, when using the application for real-time telemetry and control, you must keep the following design constraints and safety features in mind:

Performance & Packet Processing Rates

- **Maximum Frame Rate Limit:** The Qt GUI telemetry pipeline is optimized and verified to handle packet rates up to **1000 Hz (1 kHz)**. Above this rate, packet processing and UI redraw operations may consume significant CPU resources.
- **Queue Saturation (⚠ Queue Saturated):** If incoming serial data rates exceed the system's capacity, or if the OS stalls the GUI thread (e.g., during window dragging), the serial packet queue or log flush queue may saturate. When this happens, a yellow warning badge appears in the bottom status bar, and newer packets/logs will be dropped to protect system memory. **Click the badge to dismiss it.** If it reappears, the underlying condition persists – reduce the incoming data rate, increase the flush interval under **Tools** → **Logging Settings**, or close unused dock panels to free up the GUI thread.
- **Odd Byte Widths:** Decoded signal widths of 3, 5, 6, and 7 bytes are optimized using zero-copy `struct.unpack_from` combined with bitwise shifting and masking, eliminating the performance penalty of slicing and matching power-of-two decoding performance.

Data Logging & Buffer Safety

- **Decoded Log Crash Safety:** The Decoded Excel log (`*_decoded.xlsx`) is written incrementally to temporary files (`.tmp_data` and `.tmp_meta`) during the session and compiled at the end. If the

application crashes, the unsaved data is not lost; Bytehound automatically scans and prompts to recover these logs on the next launch.

- **Plot History Limits:** Plot history is stored in a fixed-capacity ring buffer (default 6,000 samples per signal). In long monitoring sessions, older points will wrap around and be discarded from memory to prevent memory leaks. Consequently, scrolling back to the beginning of a multi-hour test in the Live Plot is not supported; use the Analysis Suite on the saved log files instead.
- **Raw Console and Log line limits:** The Raw Console is capped at 3,000 blocks/lines of history, and the Activity Log is capped at 5,000 blocks. Old messages are automatically truncated from the display buffer.

Dock Window CPU Optimization (Data Gaps)

- **Hidden Dock Optimization:** To save CPU cycles on low-spec hardware, the application suspends UI rendering operations on hidden panels. However, plot history buffering remains fully active in the background for closed panels, ensuring that reopening the Live Plot displays a continuous timeline with no data gaps.

Configuration Constraints

- **Strict Config Duplicate Validations:** The configuration loader validates all sheets for duplicates. If you define duplicate entries—such as duplicate variables in a frame, duplicate `tx_commands`, duplicate `enums` values, duplicate `bit_index`/labels on the same bitfield signal, duplicate calculations, or duplicate polling schedules—the loader will raise a validation `ConfigError` rather than silently overwriting.
- **Modbus RTU Specifications:** The Modbus RTU protocol specifies big-endian encoding for register addresses and data values, and little-endian encoding for the CRC checksum. Custom `byte_order` configurations in the Protocol sheet are ignored when `parser_type = modbus_rtu` is selected.
- **Polling vs. Streaming:** Do not enable **Auto-Fetch (F10)** on a device that is already streaming data automatically (unsolicited). Sending poll requests to a streaming device will cause transmission collisions and corrupt the incoming data packets.